

Big Data

Here is an example of code that can be used for Big Data processing:

```
from pyspark import SparkContext, SparkConf
from pyspark.sql import SparkSession

conf = SparkConf().setAppName("BigData")
sc = SparkContext.getOrCreate(conf)

spark = SparkSession.builder.appName("BigData").getOrCreate()

data = spark.read.format("csv").option("header", "true").load("data.csv")

data.show()
```

Unit 1 - Introduction to Big Data

Big data refers to the large, diverse sets of information that grow at ever-increasing rates. It encompasses the volume of information, the velocity or speed at which it is created and collected, and the variety or scope of the data points being covered. Big data often comes from multiple sources and arrives in multiple formats.

The rise of big data has been driven by the growth of the internet, social media, and the increasing number of devices connected to the internet. This has led to an explosion in the amount of data being generated, stored, and analyzed.

Big data has the potential to provide valuable insights and drive innovation in many fields, including business, healthcare, and government. However, it also presents challenges in terms of data storage, management, and analysis.

To effectively work with big data, it is necessary to have the right tools and techniques for data storage, management, and analysis. This includes technologies such as Hadoop, Spark, and NoSQL databases, as well as techniques such as machine learning and data mining.

In this unit, we will explore the basics of big data, including its definition, characteristics, and applications. We will also discuss the challenges and opportunities presented by big data, and the tools and techniques used to work with it.

Types of digital data in big data

Big data refers to the large and complex sets of data that traditional data processing methods cannot handle. Digital data in big data can be categorized into several types, including:

1. **Structured data:** This type of data is organized in a predefined manner, such as in a database, and can be easily analyzed. Examples include customer names, addresses, and purchase histories.
2. **Unstructured data:** This type of data has no predefined format or organization and can be more difficult to analyze. Examples include social media posts, images, and videos.
3. **Semi-structured data:** This type of data falls between structured and unstructured data and contains elements of both. Examples include emails, which have a structured header but unstructured body content.
4. **Time-series data:** This type of data is collected over time and can be used to track changes and identify trends. Examples include stock prices, weather data, and website traffic.
5. **Geospatial data:** This type of data is associated with a specific location and can be used to create maps and analyze spatial relationships. Examples include GPS data, satellite imagery, and census data.

These are just a few examples of the types of digital data that can be found in big data. Each type of data requires different methods for storage, processing, and analysis.

History of Big Data Innovation

Big data innovation has a long history that can be traced back to the early days of computing. In the 1960s and 1970s, the development of databases and data storage technologies allowed for the storage and retrieval of large amounts of data. In the 1980s and 1990s, advances in computer hardware, such as faster processors and larger storage devices, made it possible to process and analyze larger data sets.

In the early 2000s, the growth of the internet and the proliferation of digital devices led to an explosion in the amount of data being generated. This led to the development of new technologies and techniques for processing and analyzing large data sets, such as distributed computing and machine learning.

Today, big data innovation continues to evolve, with new technologies and techniques being developed to help organizations make sense of the vast amounts of data at their disposal. These innovations are driving advances in fields such as healthcare, finance, and marketing, and are helping organizations to make more informed decisions and to operate more efficiently.

Introduction to Big Data platform

A Big Data platform is a type of IT solution that combines the features and capabilities of several Big Data application and utilities within a single solution. It is designed to handle, process, analyze and store large, complex, and

multi-structured data sets that are beyond the capabilities of traditional data processing systems.

Big Data platforms typically include a combination of the following components:

- Data storage and management systems, such as Hadoop Distributed File System (HDFS) or NoSQL databases
- Data processing and analysis tools, such as Apache Spark or Apache Flink
- Data integration and ingestion tools, such as Apache Sqoop or Apache Flume
- Data visualization and reporting tools, such as Tableau or QlikView

These components work together to provide a comprehensive solution for managing and analyzing Big Data. Big Data platforms are commonly used in industries such as finance, healthcare, retail, and telecommunications, where large amounts of data are generated and analyzed on a regular basis.

Drivers for Big Data

Here is an example of code that could be used to analyze drivers for Big Data:

```
# Import necessary libraries
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression

# Load data
data = pd.read_csv('big_data_drivers.csv')

# Define independent and dependent variables
X = data[['driver1', 'driver2', 'driver3']]
y = data['big_data_usage']

# Create and fit linear regression model
model = LinearRegression()
model.fit(X, y)

# Print coefficients
print('Driver 1 coefficient:', model.coef_[0])
print('Driver 2 coefficient:', model.coef_[1])
print('Driver 3 coefficient:', model.coef_[2])
```

Big Data Architecture

Big data architecture is the overarching system used to ingest, process, and analyze large and complex data sets. It involves the use of various technologies and tools to design a scalable and flexible infrastructure that can handle the storage and processing of big data.

Here is an example of a big data architecture using Hadoop and Spark:

```
from pyspark import SparkConf, SparkContext
from pyspark.sql import SQLContext

conf = SparkConf().setAppName("BigDataArchitecture")
sc = SparkContext.getOrCreate(conf)
sqlContext = SQLContext(sc)

# Load data from HDFS
data = sqlContext.read.format("com.databricks.spark.csv").option("header", "true").load("hdfs://namenode:9000/path/to/data.csv")

# Perform data processing and analysis
result = data.groupBy("column1").agg({"column2": "sum"})

# Save result to HDFS
result.write.format("com.databricks.spark.csv").option("header", "true").save("hdfs://namenode:9000/path/to/result.csv")
```

This code sets up a Spark context and SQL context, loads data from HDFS, performs data processing and analysis using Spark's DataFrame API, and saves the result back to HDFS. This is just one example of how big data architecture can be implemented using Hadoop and Spark. There are many other tools and technologies that can be used to design a big data architecture that meets the specific needs of an organization.

Big data characteristics

Big data is characterized by the 5 Vs: Volume, Velocity, Variety, Veracity, and Value.

- **Volume:** Big data refers to the large amount of data that is generated and collected. This data can come from various sources and can be structured or unstructured.
- **Velocity:** The speed at which data is generated and processed is also a characteristic of big data. This data is generated in real-time and needs to be processed quickly to extract value from it.
- **Variety:** Big data can come in various formats, including structured, semi-structured, and unstructured data. This data can come from various sources, including text, images, videos, and audio.
- **Veracity:** The accuracy and reliability of the data is also a characteristic of big data. This data needs to be accurate and reliable to extract value from it.
- **Value:** The value that can be extracted from the data is also a characteristic of big data. This data needs to be analyzed and processed to extract value from it.

```
def big_data_characteristics():
    characteristics = {
        'Volume': 'Large amount of data generated and collected from various sources',
        'Velocity': 'Speed at which data is generated and processed in real-time',
        'Variety': 'Data comes in various formats including structured, semi-structured, and unstructured',
        'Veracity': 'Accuracy and reliability of the data',
        'Value': 'Value that can be extracted from the data through analysis and processing'
    }
    return characteristics
```

5 Vs of Big Data

The 5 Vs of Big Data refer to the five key characteristics that define Big Data: Volume, Velocity, Variety, Veracity, and Value.

1. **Volume** refers to the massive amount of data that is generated and stored. This data can come from various sources such as social media, sensors, and business transactions.
2. **Velocity** refers to the speed at which data is generated, processed, and analyzed. With the increasing use of real-time data, the velocity of data has become an important factor in Big Data.
3. **Variety** refers to the different types of data that are generated. This can include structured data, such as numbers and dates, as well as unstructured data, such as text, images, and videos.
4. **Veracity** refers to the accuracy and reliability of the data. With the increasing amount of data being generated, it is important to ensure that the data is accurate and can be trusted.
5. **Value** refers to the ability to extract useful insights from the data. The ultimate goal of Big Data is to provide valuable insights that can be used to make better decisions.

Big Data technology components

Big Data technology is a collection of tools and techniques used to process, store, and analyze large and complex data sets. Some of the key components of Big Data technology include:

1. **Distributed file systems:** These systems allow data to be stored across multiple servers, providing scalability and fault tolerance. Examples include Hadoop Distributed File System (HDFS) and Google File System (GFS).
2. **NoSQL databases:** These databases are designed to handle large volumes of structured and unstructured data. They provide flexible data models and can scale horizontally. Examples include MongoDB, Cassandra, and Couchbase.

3. **Data processing frameworks:** These frameworks provide tools for processing and analyzing large data sets. They can handle batch processing, real-time processing, and machine learning tasks. Examples include Apache Hadoop, Apache Spark, and Apache Flink.
4. **Data visualization tools:** These tools help users to explore and understand large data sets by presenting the data in a visual format. Examples include Tableau, QlikView, and D3.js.
5. **Cloud computing platforms:** These platforms provide scalable and cost-effective infrastructure for storing and processing large data sets. Examples include Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP).

These are some of the key components of Big Data technology. Each component plays a crucial role in enabling organizations to derive insights from their data.

Big Data importance

Big data refers to the large, diverse sets of information that grow at ever-increasing rates. It encompasses the volume of information, the velocity or speed at which it is created and collected, and the variety or scope of the data points being covered. Big data often comes from multiple sources and arrives in multiple formats.

The importance of big data lies in how an organization utilizes the data collected. By analyzing large amounts of information, businesses can uncover hidden patterns, correlations, and other insights. With these insights, businesses can make more informed decisions, better understand their customers, and gain a competitive advantage.

Big data can also be used to improve operational efficiency, reduce costs, and minimize risk. For example, by analyzing large amounts of data, companies can identify areas where they can streamline processes, reduce waste, and improve overall efficiency. Additionally, big data can be used to detect and prevent fraud, as well as to improve cybersecurity.

Overall, the importance of big data lies in its ability to help organizations make better decisions, improve operations, and gain a competitive advantage. By harnessing the power of big data, businesses can unlock new opportunities and achieve greater success.

Big Data applications

Big Data applications are used to process and analyze large and complex datasets. These applications can be used in various industries such as healthcare, finance, and retail to gain insights and make data-driven decisions. Here is an example of a code that can be used to analyze a large dataset using the Apache Hadoop framework:

```

from pyspark import SparkContext, SparkConf
from pyspark.sql import SQLContext

# Set up the Spark configuration and context
conf = SparkConf().setAppName("BigDataApp")
sc = SparkContext.getOrCreate(conf)
sqlContext = SQLContext(sc)

# Load the data from HDFS
data = sqlContext.read.format("com.databricks.spark.csv").option("header", "true").load("hdi

# Perform data analysis
data.groupBy("column1").count().show()

```

Big Data features – security, compliance, auditing and protection

```

class BigData:
    def __init__(self, security, compliance, auditing, protection):
        self.security = security
        self.compliance = compliance
        self.auditing = auditing
        self.protection = protection

    def set_security(self, security):
        self.security = security

    def set_compliance(self, compliance):
        self.compliance = compliance

    def set_auditing(self, auditing):
        self.auditing = auditing

    def set_protection(self, protection):
        self.protection = protection

    def get_security(self):
        return self.security

    def get_compliance(self):
        return self.compliance

    def get_auditing(self):
        return self.auditing

    def get_protection(self):
        return self.protection

```

Security of Big Data

Here is an example of code that can be used to implement security measures for Big Data:

```
from pyspark import SparkConf, SparkContext
from pyspark.sql import SQLContext
from pyspark.sql.functions import *

conf = SparkConf().setAppName("BigDataSecurity")
sc = SparkContext.getOrCreate(conf)
sqlContext = SQLContext(sc)

# Load data into a DataFrame
data = sqlContext.read.format("com.databricks.spark.csv").option("header", "true").load("data.csv")

# Encrypt sensitive data
data = data.withColumn("encrypted_column", encrypt(col("sensitive_column")))

# Save encrypted data
data.write.format("com.databricks.spark.csv").option("header", "true").save("encrypted_data.csv")
```

Compliance of Big Data

Compliance in the context of big data refers to the adherence to laws, regulations, and standards that govern the collection, storage, and use of large amounts of data. Here is an example of code that checks for compliance with the General Data Protection Regulation (GDPR) when handling user data:

```
def check_gdpr_compliance(user_data):
    # Check if user has given consent for data processing
    if not user_data['consent']:
        return False
    # Check if data is being processed for a specific, explicit, and legitimate purpose
    if not user_data['purpose']:
        return False
    # Check if data is being processed in a transparent manner
    if not user_data['transparency']:
        return False
    # Check if data is being processed in a way that ensures its security
    if not user_data['security']:
        return False
    return True
```

This code checks if the user has given consent for data processing, if the data is being processed for a specific, explicit, and legitimate purpose, if the data is being processed in a transparent manner, and if the data is being processed in a way that ensures its security. If any of these conditions are not met, the

function returns `False`, indicating non-compliance with the GDPR. Otherwise, the function returns `True`, indicating compliance with the GDPR.

Auditing of Big Data

Here is an example of code for auditing Big Data:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

# Create a Spark session
spark = SparkSession.builder.appName("BigDataAudit").getOrCreate()

# Load data from HDFS
data = spark.read.format("csv").option("header", "true").load("hdfs://path/to/data.csv")

# Define audit function
def audit_data(data):
    # Count number of rows
    row_count = data.count()
    print(f"Number of rows: {row_count}")

    # Count number of null values in each column
    null_counts = data.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in
                                data.columns]).collect()
    print("Number of null values in each column:")
    null_counts.show()

    # Compute summary statistics for numeric columns
    summary = data.describe()
    print("Summary statistics for numeric columns:")
    summary.show()

# Run audit on data
audit_data(data)
```

Protection of Big Data

Here is an example of code that can be used to protect Big Data:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import sha2, concat_ws

# Create a Spark session
spark = SparkSession.builder.appName("Protecting Big Data").getOrCreate()

# Load data
data = spark.read.format("csv").option("header", "true").load("data.csv")
```

```

# Hash sensitive columns
data = data.withColumn("hashed_name", sha2(concat_ws(" ", data["first_name"], data["last_name"]), 256))
data = data.withColumn("hashed_email", sha2(data["email"], 256))

# Drop original sensitive columns
data = data.drop("first_name", "last_name", "email")

# Save protected data
data.write.format("csv").option("header", "true").save("protected_data.csv")

```

This code uses the PySpark library to load data from a CSV file, hash sensitive columns (such as name and email) using the SHA-256 algorithm, drop the original sensitive columns, and save the protected data to a new CSV file. This is one way to protect Big Data by anonymizing sensitive information.

Big Data privacy

Big Data privacy refers to the measures taken to protect the confidentiality and security of personal information collected, stored, and analyzed through Big Data techniques. Here is an example of a code that can be used to implement privacy measures in a Big Data system:

```

from pyspark.sql.functions import sha2
from pyspark.sql import SparkSession

# Create a Spark session
spark = SparkSession.builder.appName("BigDataPrivacy").getOrCreate()

# Load data into a DataFrame
data = spark.read.format("csv").option("header", "true").load("data.csv")

# Hash sensitive columns to protect privacy
data = data.withColumn("hashed_name", sha2(data["name"], 256))
data = data.withColumn("hashed_email", sha2(data["email"], 256))

# Drop original sensitive columns
data = data.drop("name", "email")

# Save the data with privacy measures applied
data.write.format("csv").option("header", "true").save("data_privacy.csv")

```

This code uses the `sha2` function from the `pyspark.sql.functions` module to hash sensitive columns such as `name` and `email` in a `DataFrame`. The original sensitive columns are then dropped and the data is saved with the privacy measures applied. This is just one example of how privacy can be implemented in a Big Data system. There are many other techniques and methods that can

be used to protect the privacy of personal information in Big Data.

Big Data ethics

Big Data ethics refers to the ethical considerations surrounding the collection, analysis, and use of large amounts of data. It is important to ensure that the data is collected and used in a responsible and transparent manner, respecting the privacy and rights of individuals. Here is an example of a code of ethics for Big Data:

```
class BigDataEthics:
    def __init__(self, data):
        self.data = data

    def responsible_collection(self):
        # Ensure that data is collected in a responsible and transparent manner
        pass

    def respect_privacy(self):
        # Respect the privacy and rights of individuals
        pass

    def transparent_use(self):
        # Ensure that data is used in a transparent and responsible manner
        pass
```

Big Data Analytics

Big Data Analytics involves the process of examining large and varied data sets to uncover hidden patterns, unknown correlations, market trends, customer preferences, and other useful information. Here is an example of a code that can be used for Big Data Analytics using Python:

```
import pandas as pd
from sklearn import preprocessing
from sklearn.cluster import KMeans

# Load the data
data = pd.read_csv('data.csv')

# Preprocess the data
data = preprocessing.scale(data)

# Create a kmeans model
kmeans = KMeans(n_clusters=3)
kmeans.fit(data)
```

```
# Get the cluster assignments
labels = kmeans.labels_
```

Challenges of conventional systems compared to Big Data

Conventional systems are designed to handle structured data, which is data that is organized in a predefined manner. However, with the rise of Big Data, there has been an increase in the amount of unstructured data, which is data that is not organized in a predefined manner. This presents a challenge for conventional systems, as they are not designed to handle unstructured data.

Another challenge of conventional systems compared to Big Data is the volume of data. Big Data involves dealing with large amounts of data, which can be difficult for conventional systems to handle. This is because conventional systems are not designed to scale to handle large amounts of data.

Finally, the velocity of data is another challenge for conventional systems compared to Big Data. Big Data involves dealing with data that is being generated at a high rate. Conventional systems may not be able to keep up with the rate at which data is being generated, which can result in data loss or delays in processing.

In summary, the challenges of conventional systems compared to Big Data include handling unstructured data, dealing with large volumes of data, and keeping up with the velocity of data. These challenges can be addressed by using systems that are specifically designed to handle Big Data.

Intelligent Data Analysis in Big Data

Here is an example of code for intelligent data analysis in Big Data using Python:

```
import pandas as pd
from sklearn import preprocessing
from sklearn.cluster import KMeans

# Load the data
data = pd.read_csv('big_data.csv')

# Preprocess the data
data = preprocessing.scale(data)

# Apply KMeans clustering
kmeans = KMeans(n_clusters=3)
kmeans.fit(data)

# Get the cluster labels
labels = kmeans.labels_
```

Nature of Data in Big Data

Big data refers to the large, diverse sets of information that grow at ever-increasing rates. It encompasses the volume of information, the velocity or speed at which it is created and collected, and the variety or scope of the data points being covered. Big data often comes from multiple sources and arrives in multiple formats.

The nature of data in big data is characterized by the 3Vs: Volume, Velocity, and Variety.

- **Volume:** The amount of data being generated and stored is massive and continues to grow exponentially. This data comes from various sources such as social media, business transactions, and machine-generated data.
- **Velocity:** The speed at which data is generated, processed, and analyzed is increasing rapidly. This is due to the growth of real-time data processing and the Internet of Things (IoT).
- **Variety:** Data comes in various formats, including structured, semi-structured, and unstructured data. Structured data is data that is organized into a specific format or schema, such as a database. Semi-structured data is data that has some structure but is not organized into a specific format, such as a JSON file. Unstructured data is data that has no specific format or structure, such as text or images.

These characteristics of big data present challenges in terms of storage, processing, and analysis. However, with the right tools and techniques, big data can provide valuable insights and drive better decision-making.

Analytic Processes and Tools for Big Data

There are several analytic processes and tools available for handling big data. Some of the most commonly used tools include:

- **Hadoop:** An open-source framework for storing and processing large datasets using distributed computing.
- **Spark:** An open-source data processing engine that can handle large-scale data processing tasks.
- **NoSQL databases:** Non-relational databases that can handle large volumes of structured and unstructured data.
- **Machine learning algorithms:** Algorithms that can learn from data and make predictions or decisions based on that data.

These tools and processes can be used to perform a variety of analytic tasks, such as data mining, predictive modeling, and data visualization. The specific tools and processes used will depend on the specific needs and goals of the analysis.

Example code for using Hadoop to process big data

```

from pyspark import SparkContext, SparkConf

conf = SparkConf().setAppName("BigDataAnalytics")
sc = SparkContext.getOrCreate(conf)

# Load data from HDFS
data = sc.textFile("hdfs://path/to/data")

# Perform analysis
result = data.map(lambda x: x.split()).filter(lambda x: len(x) > 0).count()

# Save result to HDFS
result.saveAsTextFile("hdfs://path/to/result")

```

Analysis vs Reporting in Big Data

Analysis and reporting are two important aspects of working with big data. Analysis refers to the process of examining, cleaning, transforming, and modeling data to discover useful information, draw conclusions, and support decision-making. Reporting, on the other hand, refers to the process of organizing data into informational summaries to monitor how different areas of a business are performing.

In the context of big data, analysis often involves using advanced analytical techniques and tools to process large and complex datasets. This can include machine learning, data mining, and predictive analytics. Reporting, on the other hand, often involves creating visualizations and dashboards to present data in an easily understandable format.

Both analysis and reporting are important for making data-driven decisions. Analysis helps to uncover insights and patterns in the data, while reporting helps to communicate these insights to others. Together, they can help organizations to make better decisions and improve their operations.

Modern Data Analytic Tools for Big Data

There are several modern data analytic tools available for handling big data. Some of the most popular tools include:

1. **Apache Hadoop:** An open-source framework for distributed storage and processing of large datasets.
2. **Apache Spark:** An open-source, fast, and general-purpose cluster computing system for big data processing.
3. **NoSQL Databases:** Non-relational databases such as MongoDB, Cassandra, and Couchbase, designed to handle large volumes of structured and unstructured data.

4. **Python:** A popular programming language with a rich ecosystem of libraries and tools for data analysis and machine learning.
5. **R:** A programming language and software environment for statistical computing and graphics.

These tools, among others, provide powerful capabilities for managing, processing, and analyzing big data. It is important to choose the right tool for the specific needs of the project.

Unit 2 - Hadoop and Map Reduce

Hadoop is an open-source software framework for storing and processing large datasets using a distributed computing model. It is designed to scale up from single servers to thousands of machines, each offering local computation and storage. The core of Hadoop consists of a storage part, known as Hadoop Distributed File System (HDFS), and a processing part called MapReduce.

MapReduce is a programming model for processing large datasets in parallel across a Hadoop cluster. It consists of two main phases: the Map phase and the Reduce phase. In the Map phase, the input data is divided into chunks and processed by multiple map tasks in parallel. Each map task processes a chunk of data and produces a set of intermediate key-value pairs. In the Reduce phase, the intermediate key-value pairs are grouped by key and processed by multiple reduce tasks in parallel. Each reduce task processes a group of key-value pairs with the same key and produces a set of output values.

Here is an example of a simple MapReduce program that counts the number of occurrences of each word in a text file:

```
from mrjob.job import MRJob

class MRWordCount(MRJob):

    def mapper(self, _, line):
        # split the line into words
        words = line.split()
        # emit each word as a key with a value of 1
        for word in words:
            yield (word, 1)

    def reducer(self, key, values):
        # sum the values for each key
        yield (key, sum(values))

if __name__ == '__main__':
    MRWordCount.run()
```

This code can be run on a Hadoop cluster using the `mrjob` library. The `mapper`

function takes a line of text as input and emits each word in the line as a key with a value of 1. The **reducer** function takes a key (a word) and a list of values (the counts) as input and emits the sum of the values for each key. This produces the final word count for each word in the text file.

Hadoop

Here is an example of a simple Hadoop MapReduce program written in Java:

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable>{

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context
            ) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {
        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values,
            Context context
            ) throws IOException, InterruptedException {

            int sum = 0;
```



```

        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

This program counts the occurrences of each word in a given input file and outputs the results to an output file. The `TokenizerMapper` class tokenizes the input text and emits each word with a value of 1. The `IntSumReducer` class sums up the values for each word and emits the final count for each word.

History of Hadoop

Hadoop was started by Doug Cutting and Mike Cafarella in the year 2002 when they both started to work on the Apache Nutch project. The Apache Nutch project was the process of building a search engine system that could index 1 billion pages . Hadoop was named after an extinct species of mammoth, a so-called Yellow Hadoop . It was created by the Apache Software Foundation in 2006, based on a white paper written by Google in 2003 that described the Google File System (GFS) and the MapReduce programming model . In 2008, Hadoop defeated the supercomputers and became the fastest system on the planet for sorting terabytes of data .

Apache Hadoop

Apache Hadoop is an open-source software framework for distributed storage and processing of large datasets. Here is an example of how to run a MapReduce job using Hadoop:

```
import org.apache.hadoop.conf.Configuration;
```

```

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable>{

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context
            ) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {
        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values,
            Context context
            ) throws IOException, InterruptedException {

            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");
    }
}

```

```

        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

This code defines a MapReduce job that counts the occurrences of words in a text file. The `TokenizerMapper` class tokenizes the input text and emits a key-value pair for each word, where the key is the word and the value is 1. The `IntSumReducer` class sums up the values for each key and emits a key-value pair where the key is the word and the value is the total count. The `main` method sets up the job configuration and runs the job.

Hadoop Distributed File System

Here is an example of code for Hadoop Distributed File System (HDFS) written in Java:

```

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;

import java.io.IOException;

public class HDFSExample {
    public static void main(String[] args) throws IOException {
        Configuration conf = new Configuration();
        conf.set("fs.defaultFS", "hdfs://localhost:9000");
        FileSystem fs = FileSystem.get(conf);
        Path path = new Path("/user/hadoop/file.txt");
        if (fs.exists(path)) {
            System.out.println("File exists");
        } else {
            System.out.println("File does not exist");
        }
        fs.close();
    }
}

```

This code creates a `Configuration` object and sets the Hadoop file system to be HDFS with the `fs.defaultFS` property. It then creates a `FileSystem` object

using the configuration and checks if a file exists at the specified path on the HDFS. If the file exists, it prints “File exists” to the console, otherwise it prints “File does not exist”. Finally, it closes the `FileSystem` object.

Components of Hadoop

Hadoop is an open-source software framework for storing and processing large datasets. It is designed to scale up from a single server to thousands of machines, each offering local computation and storage. The core components of Hadoop are:

1. **Hadoop Distributed File System (HDFS):** A distributed file system that provides high-throughput access to application data.
2. **Hadoop MapReduce:** A programming model for large scale data processing.
3. **Hadoop YARN:** A framework for job scheduling and cluster resource management.
4. **Hadoop Common:** A set of common utilities that support the other Hadoop modules.

data format co

Code for data format co

Analyzing Data with Hadoop

Hadoop is an open-source software framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models. It is designed to scale up from single servers to thousands of machines, each offering local computation and storage.

Here are some key points to consider when analyzing data with Hadoop:

1. **Data Storage:** Hadoop uses the Hadoop Distributed File System (HDFS) to store data across multiple machines. This allows for efficient data storage and retrieval, as well as fault tolerance in case of machine failure.
2. **Data Processing:** Hadoop uses the MapReduce programming model to process data. This model involves dividing the data into smaller chunks, processing each chunk in parallel, and then combining the results.
3. **Scalability:** Hadoop is designed to scale up from a single machine to thousands of machines, allowing for the efficient processing of large data sets.
4. **Flexibility:** Hadoop can handle structured, semi-structured, and unstructured data, making it a versatile tool for data analysis.

5. **Cost-effectiveness:** Hadoop is an open-source software, meaning it is free to use. Additionally, it can run on commodity hardware, reducing the cost of data analysis.

In summary, Hadoop is a powerful tool for analyzing large data sets. Its distributed architecture, scalability, flexibility, and cost-effectiveness make it a popular choice for data analysis.

Scaling out with Hadoop

Hadoop is an open-source software framework for distributed storage and processing of large datasets. It is designed to scale out from a single server to thousands of machines, each offering local computation and storage. Here is an example of how to scale out with Hadoop:

```
from pyspark import SparkConf, SparkContext

conf = SparkConf().setAppName("ScaleOut").setMaster("local")
sc = SparkContext.getOrCreate(conf)

data = [1, 2, 3, 4, 5]
distData = sc.parallelize(data)

result = distData.reduce(lambda a, b: a + b)
print(result)
```

This code creates a SparkConf object and sets the application name and master URL. The master URL is set to “local” to run the application locally. The SparkContext is then created using the configuration. The data is then parallelized using the `parallelize` method, which distributes the data across the cluster. The `reduce` method is then used to aggregate the data and compute the result. The result is then printed to the console.

Hadoop Streaming

Hadoop Streaming is a utility that comes with the Hadoop distribution. This utility allows you to create and run Map/Reduce jobs with any executable or script as the mapper and/or the reducer. Here is an example of how to use Hadoop Streaming with Python:

```
# mapper.py
import sys
for line in sys.stdin:
    line = line.strip()
    words = line.split()
    for word in words:
        print('%s\t%s' % (word, 1))
```

```

# reducer.py
from operator import itemgetter
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word, count = line.split('\t', 1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
            current_count = count
            current_word = word
if current_word == word:
    print('%s\t%s' % (current_word, current_count))

```

To run the MapReduce job, you would use the following command:

```

bin/hadoop jar contrib/streaming/hadoop-streaming-*.jar \
-input myInputDirs \
-output myOutputDir \
-mapper /path/to/mapper.py \
-reducer /path/to/reducer.py

```

Hadoop Pipes

Hadoop Pipes is a C++ API to implement MapReduce applications. Here is an example of a simple word count program using Hadoop Pipes:

```

#include <algorithm>
#include <string>
#include <hadoop/Pipes.hh>
#include <hadoop/TemplateFactory.hh>
#include <hadoop/StringUtils.hh>

class WordCountMap: public HadoopPipes::Mapper {
public:
    WordCountMap(HadoopPipes::TaskContext& context){}
    void map(HadoopPipes::MapContext& context) {
        std::vector<std::string> words = HadoopUtils::splitString(context.getInputValue(), " ");
    }
}

```

```

        for(unsigned int i=0; i < words.size(); ++i) {
            context.emit(words[i], "1");
        }
    }
};

class WordCountReduce: public HadoopPipes::Reducer {
public:
    WordCountReduce(HadoopPipes::TaskContext& context){}
    void reduce(HadoopPipes::ReduceContext& context) {
        int count = 0;
        while (context.nextValue()) {
            count += HadoopUtils::toInt(context.getInputValue());
        }
        context.emit(context.getInputKey(), HadoopUtils::toString(count));
    }
};

int main(int argc, char *argv[]) {
    return HadoopPipes::runTask(HadoopPipes::TemplateFactory<WordCountMap, WordCountReduce>())
}

```

Hadoop Echo System

Hadoop is an open-source software framework for storing and processing large datasets. The Hadoop ecosystem includes a number of related technologies that can be used together to build a complete big data solution. Here is an example of how to set up a Hadoop cluster:

1. Install Hadoop on a cluster of machines.
2. Configure the Hadoop Distributed File System (HDFS) to store data across the cluster.
3. Set up the YARN resource manager to manage the allocation of resources for processing tasks.
4. Use MapReduce or another processing framework to write and run data processing jobs.
5. Use tools such as Hive or Pig to write and run queries against the data stored in HDFS.

This is just one example of how to set up a Hadoop ecosystem. There are many other tools and technologies that can be used in conjunction with Hadoop to build a complete big data solution.

Map Reduce

MapReduce is a programming model for processing large data sets with a parallel, distributed algorithm on a cluster. Here is an example of a simple MapRe-

duce program in Python that counts the occurrences of words in a text file:

```
from collections import defaultdict
import sys

def map_function(document):
    words = document.split()
    for word in words:
        yield (word, 1)

def reduce_function(key, values):
    yield (key, sum(values))

def main():
    intermediate = defaultdict(list)
    for line in sys.stdin:
        for key, value in map_function(line):
            intermediate[key].append(value)

    for key, values in intermediate.items():
        for result in reduce_function(key, values):
            print(result)

if __name__ == '__main__':
    main()
```

Map Reduce framework and basics

MapReduce is a programming model and an associated implementation for processing and generating large data sets. Users specify a map function that processes a key/value pair to generate a set of intermediate key/value pairs, and a reduce function that merges all intermediate values associated with the same intermediate key.

Here is an example of a simple MapReduce program in Python that counts the number of occurrences of each word in a text file:

```
from mrjob.job import MRJob

class MRWordFrequencyCount(MRJob):

    def mapper(self, _, line):
        # yield each word in the line
        for word in line.split():
            yield (word.lower(), 1)

    def reducer(self, key, values):
```



```

        # sum the words we've seen so far
        yield (key, sum(values))

if __name__ == '__main__':
    MRWordFrequencyCount.run()

```

This code defines a MapReduce job that consists of a mapper function and a reducer function. The mapper function takes a line of text as input and outputs a key-value pair for each word in the line, where the key is the word and the value is 1. The reducer function takes a key and a list of values as input and outputs the sum of the values for that key, which represents the number of occurrences of the word in the text file.

How Map Reduce works

MapReduce is a programming model for processing large data sets with a parallel, distributed algorithm on a cluster. It consists of two main functions: Map and Reduce.

The Map function takes an input pair and produces a set of intermediate key/value pairs. The MapReduce framework then shuffles the intermediate data, grouping values with the same key together, and feeds them to the Reduce function.

The Reduce function accepts an intermediate key and a set of values for that key. It then merges the values to form a smaller set of values, typically zero or one output value per Reduce invocation.

Here is an example of a MapReduce program in Python that counts the number of occurrences of each word in a text file:

```

from mrjob.job import MRJob

class MRWordFrequencyCount(MRJob):

    def mapper(self, _, line):
        # split the line into words
        words = line.split()
        # emit each word with a count of 1
        for word in words:
            yield (word, 1)

    def reducer(self, key, values):
        # sum the counts for each word
        yield (key, sum(values))

if __name__ == '__main__':
    MRWordFrequencyCount.run()

```

This code defines a MapReduce job that consists of a mapper and a reducer function. The mapper function splits each line into words and emits each word with a count of 1. The reducer function then sums the counts for each word and emits the final word count. This program can be run on a Hadoop cluster to process large text files in parallel.

Developing a Map Reduce application

Here is an example of developing a Map Reduce application in Java using the Hadoop framework:

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable>{

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context
            ) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {
        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values,
            Context context
```

```

        ) throws IOException, InterruptedException {
    int sum = 0;
    for (IntWritable val : values) {
        sum += val.get();
    }
    result.set(sum);
    context.write(key, result);
}
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

This is a simple example of a Map Reduce application that counts the number of occurrences of each word in a given input text file. The `TokenizerMapper` class tokenizes the input text and emits a key-value pair for each word with the word as the key and the value as 1. The `IntSumReducer` class then sums up the values for each key and emits the final key-value pair with the word as the key and the sum as the value.

Unit tests with MRUnit

Here is an example of how to write unit tests for a MapReduce job using MRUnit:

```

import org.apache.hadoop.io.*;
import org.apache.hadoop.mrunit.mapreduce.MapDriver;
import org.junit.Before;
import org.junit.Test;

public class WordCountTest {
    MapDriver<LongWritable, Text, Text, IntWritable> mapDriver;

    @Before
    public void setUp() {

```

```

    WordCountMapper mapper = new WordCountMapper();
    mapDriver = MapDriver.newMapDriver(mapper);
}

@Test
public void testMapper() {
    mapDriver.withInput(new LongWritable(1), new Text("cat cat dog"));
    mapDriver.withOutput(new Text("cat"), new IntWritable(1));
    mapDriver.withOutput(new Text("cat"), new IntWritable(1));
    mapDriver.withOutput(new Text("dog"), new IntWritable(1));
    mapDriver.runTest();
}
}

```

This code tests the `WordCountMapper` class, which is a mapper for a word count MapReduce job. The `setUp` method initializes the `mapDriver` object with an instance of the `WordCountMapper` class. The `testMapper` method then uses the `mapDriver` object to test the mapper with an input key-value pair and checks if the output key-value pairs match the expected output.

Test Data and Local Tests in MapReduce

Here is an example of how you can create test data and perform local tests in MapReduce:

```

from mrjob.job import MRJob
from mrjob.step import MRStep
import random

class MRTest(MRJob):

    def configure_args(self):
        super(MRTest, self).configure_args()
        self.add_passthru_arg('--test', action='store_true', help='Run local tests')

    def steps(self):
        return [
            MRStep(mapper=self.mapper,
                  reducer=self.reducer)
        ]

    def mapper(self, _, line):
        # Your mapper code here
        pass

    def reducer(self, key, values):
        # Your reducer code here

```

```

        pass

    def run_tests(self):
        # Create test data
        test_data = []
        for i in range(100):
            test_data.append(str(random.randint(0, 100)))

        # Run local tests
        self.sandbox(stdin=test_data)
        with self.make_runner() as runner:
            runner.run()
            for key, value in self.parse_output(runner.cat_output()):
                print(key, value)

if __name__ == '__main__':
    MRTest().run()

```

To run local tests, you can use the `--test` command line argument when running the script:

```
python mrtest.py --test
```

This will create test data and run the MapReduce job locally using the test data as input. You can then verify the output to ensure that your code is working as expected.

Anatomy of a Map Reduce job run

A MapReduce job run consists of the following steps:

1. **Input:** The input data is divided into splits, which are logical chunks of the input data. Each split is then assigned to a map task.
2. **Map:** The map function is applied to each split, which processes the data and produces a set of intermediate key-value pairs.
3. **Shuffle:** The intermediate key-value pairs are then shuffled, which means they are redistributed across the reducers based on the key.
4. **Reduce:** The reduce function is applied to the intermediate key-value pairs with the same key, which processes the data and produces the final output.
5. **Output:** The final output is written to the specified output location.

Here is an example of a simple MapReduce job in Java:

```

public class WordCount {
    public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {
        private final static IntWritable one = new IntWritable(1);

```

```

        private Text word = new Text();
        public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
        private IntWritable result = new IntWritable();
        public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

Failures in MapReduce

MapReduce is a programming model for processing large data sets. It is designed to be fault-tolerant and to handle failures gracefully. Here is an example of how failures can be handled in a MapReduce job:

```

from mrjob.job import MRJob

class MRFailureExample(MRJob):

    def mapper(self, _, line):

```

```

    try:
        # Code that may cause an exception
        pass
    except Exception as e:
        # Handle the exception
        pass

def reducer(self, key, values):
    try:
        # Code that may cause an exception
        pass
    except Exception as e:
        # Handle the exception
        pass

if __name__ == '__main__':
    MRFailureExample.run()

```

In this example, the `mapper` and `reducer` functions are wrapped in `try/except` blocks to catch any exceptions that may occur. When an exception is caught, the code in the `except` block is executed to handle the exception. This can include logging the error, skipping the current record, or taking other appropriate actions.

Job scheduling in MapReduce

Here is an example of a simple job scheduling algorithm in MapReduce:

```

from mrjob.job import MRJob
from mrjob.step import MRStep

class MRJobScheduling(MRJob):
    def steps(self):
        return [
            MRStep(mapper=self.mapper,
                  reducer=self.reducer)
        ]

    def mapper(self, _, line):
        # extract relevant data from input
        job_id, priority, duration = line.split()
        priority = int(priority)
        duration = int(duration)
        yield priority, (job_id, duration)

    def reducer(self, key, values):
        # sort jobs by priority

```

```

        sorted_jobs = sorted(values, key=lambda x: x[0])
        current_time = 0
        for job in sorted_jobs:
            job_id, duration = job
            start_time = current_time
            end_time = current_time + duration
            current_time = end_time
            yield job_id, (start_time, end_time)

if __name__ == '__main__':
    MRJobScheduling.run()

```

This code defines a MapReduce job that takes as input a list of jobs, each with an ID, priority, and duration. The mapper extracts the relevant data from the input and yields the priority as the key and the job ID and duration as the value. The reducer then sorts the jobs by priority and schedules them one by one, keeping track of the current time and calculating the start and end times for each job. The output is the job ID and the start and end times for each job.

Shuffle and Sort in MapReduce

In MapReduce, the shuffle and sort phase occurs between the map and reduce phases. During this phase, the output from the map phase is shuffled and sorted before being sent to the reduce phase.

Here is an example of how shuffle and sort can be implemented in MapReduce using Python:

```

from itertools import groupby
from operator import itemgetter

def shuffle_sort(map_output):
    # Sort the map output by key
    sorted_map_output = sorted(map_output, key=itemgetter(0))
    # Group the sorted map output by key
    grouped_map_output = groupby(sorted_map_output, key=itemgetter(0))
    # Return the grouped map output
    return grouped_map_output

```

Task Execution in Map Reduce

Here is an example of how task execution can be implemented in MapReduce using Python:

```

from mrjob.job import MRJob

class MRTaskExecution(MRJob):
    def mapper(self, _, line):

```



```

        # Mapper code here
    pass

    def reducer(self, key, values):
        # Reducer code here
    pass

if __name__ == '__main__':
    MRTaskExecution.run()

```

This code defines a MapReduce job using the `mrjob` library. The `mapper` function takes in a key-value pair (in this case, the key is ignored and the value is a line of text) and outputs intermediate key-value pairs. The `reducer` function takes in a key and a list of values and outputs the final key-value pairs.

Map Reduce types in map reduce

MapReduce is a programming model for processing large data sets with a parallel, distributed algorithm on a cluster. There are two main types of operations in the MapReduce model: Map and Reduce.

The Map operation takes an input key-value pair and produces a set of intermediate key-value pairs. The Map function is applied to each input key-value pair and the resulting intermediate key-value pairs are grouped by key and passed to the Reduce operation.

The Reduce operation takes an intermediate key and a set of values for that key and produces a set of output key-value pairs. The Reduce function is applied to each intermediate key and its set of values to produce the final output.

Here is an example of a simple MapReduce program in Python that counts the number of occurrences of each word in a text file:

```

from mrjob.job import MRJob

class MRWordCount(MRJob):

    def mapper(self, _, line):
        for word in line.split():
            yield (word, 1)

    def reducer(self, key, values):
        yield (key, sum(values))

if __name__ == '__main__':
    MRWordCount.run()

```

In this example, the `mapper` function takes each line of the input text file and splits it into words. For each word, it produces a key-value pair with the word

as the key and the value 1. The **reducer** function takes each key (word) and its set of values (a list of 1s) and produces a key-value pair with the word as the key and the sum of the values as the value, which is the count of the occurrences of the word in the text file.

Input Formats in MapReduce

In MapReduce, the input data is divided into splits, which are then processed by the map tasks. The **InputFormat** class is responsible for defining how the input data is split and read. There are several built-in **InputFormat** classes in Hadoop, including:

- **TextInputFormat**: This is the default **InputFormat** for MapReduce jobs. It reads data line by line, where each line is a key-value pair. The key is the byte offset of the line, and the value is the content of the line.
- **KeyValueTextInputFormat**: This **InputFormat** reads data line by line, where each line is a key-value pair separated by a delimiter (by default, a tab character).
- **SequenceFileInputFormat**: This **InputFormat** reads data from a sequence file, which is a binary file format that stores key-value pairs.
- **NLineInputFormat**: This **InputFormat** reads data line by line, where *N* lines are grouped into a single split and processed by a single map task.

Here is an example of how to set the **InputFormat** for a MapReduce job in Java:

```
Job job = Job.getInstance(conf, "MyJob");
job.setInputFormatClass(TextInputFormat.class);
```

You can also create your own custom **InputFormat** by extending the **InputFormat** class and overriding the **getSplits** and **createRecordReader** methods. This allows you to define your own logic for splitting and reading the input data.

Output Formats in MapReduce

In Hadoop MapReduce, the output of the reduce task is written to the **OutputFormat** defined in the job configuration. The default **OutputFormat** is **TextOutputFormat**, which writes the output as text files in the output directory specified in the job configuration.

Here is an example of how to set the **OutputFormat** in the job configuration:

```
Job job = Job.getInstance(conf, "MyJob");
job.setOutputFormatClass(TextOutputFormat.class);
```

Other available **OutputFormats** include **SequenceFileOutputFormat**, which writes the output as Hadoop **SequenceFiles**, and **NullOutputFormat**, which discards the output.

To implement a custom `OutputFormat`, you can extend the `OutputFormat` class and override the `getRecordWriter` method to provide a custom `RecordWriter` implementation. The `RecordWriter` is responsible for writing the output data to the final output files.

Here is an example of a custom `OutputFormat` implementation:

```
public class MyOutputFormat extends OutputFormat<Text, IntWritable> {
    @Override
    public RecordWriter<Text, IntWritable> getRecordWriter(TaskAttemptContext context) {
        // return a custom RecordWriter implementation
    }
}
```

To use the custom `OutputFormat`, set it in the job configuration as shown above:

```
Job job = Job.getInstance(conf, "MyJob");
job.setOutputFormatClass(MyOutputFormat.class);
```

Map Reduce features

MapReduce is a programming model and an associated implementation for processing and generating large data sets. It has the following features:

- **Scalability:** MapReduce can process large amounts of data by dividing the work among multiple nodes in a cluster.
- **Fault tolerance:** MapReduce can handle node failures by reassigning the work to other nodes.
- **Data locality:** MapReduce tries to move the computation to the data, rather than moving the data to the computation. This reduces the amount of data that needs to be transferred over the network.
- **Simplicity:** MapReduce provides a simple programming model that abstracts away many of the complexities of distributed computing.

Here is an example of a simple MapReduce program in Python that counts the number of occurrences of each word in a text file:

```
from mrjob.job import MRJob

class MRWordFrequencyCount(MRJob):

    def mapper(self, _, line):
        # yield each word in the line
        for word in line.split():
            yield (word.lower(), 1)

    def reducer(self, key, values):
        # sum the values for each word
```

```

        yield (key, sum(values))

if __name__ == '__main__':
    MRWordFrequencyCount.run()

```

Real-world Map Reduce

MapReduce is a programming model for processing large data sets with a parallel, distributed algorithm on a cluster. Here is an example of a simple MapReduce program that counts the number of occurrences of each word in a given input set:

```

from mrjob.job import MRJob

class MRWordFrequencyCount(MRJob):

    def mapper(self, _, line):
        # yield each word in the line
        for word in line.split():
            yield word.lower(), 1

    def reducer(self, key, values):
        # sum the values for each word
        yield key, sum(values)

if __name__ == '__main__':
    MRWordFrequencyCount.run()

```

This code can be run on a Hadoop cluster or locally on a single machine. The `mapper` function takes each line of the input data and yields a key-value pair for each word in the line, with the word as the key and the value as 1. The `reducer` function takes the key-value pairs from the `mapper` function and sums the values for each key, yielding the total count for each word.

Unit 4 - HDFS (Hadoop Distributed File System) and Hadoop Environment

1. **HDFS (Hadoop Distributed File System)** is the primary storage system used by Hadoop applications.
2. HDFS is a distributed file system that provides high-throughput access to application data and is designed to be deployed on low-cost hardware.
3. HDFS is highly fault-tolerant and is designed to be deployed on large clusters of commodity hardware.
4. HDFS provides interfaces for applications to move themselves closer to where the data is located, improving data access times.
5. HDFS stores large files across multiple machines and achieves reliability by replicating the data across multiple hosts.

6. Hadoop Environment consists of Hadoop's core components, which are HDFS, MapReduce, and YARN, as well as a range of related projects such as Apache Pig, Apache Hive, Apache HBase, and others.
7. Hadoop Environment is designed to scale up from a single server to thousands of machines, with a very high degree of fault tolerance.
8. Hadoop Environment is used for processing large data sets in parallel across a large number of nodes.
9. Hadoop Environment is widely used in big data analytics, data warehousing, and other data-intensive applications.

HDFS

Hadoop Distributed File System (HDFS) is a distributed file system designed to run on commodity hardware. It is highly fault-tolerant and is designed to be deployed on low-cost hardware. Here is an example of how to write data to HDFS using the Hadoop API in Java:

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import java.io.BufferedWriter;
import java.io.OutputStreamWriter;

public class HdfsWrite {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        conf.set("fs.defaultFS", "hdfs://localhost:9000");
        FileSystem fs = FileSystem.get(conf);
        Path path = new Path("/user/hadoop/test.txt");
        BufferedWriter writer = new BufferedWriter(new OutputStreamWriter(fs.create(path)));
        writer.write("Hello, HDFS!");
        writer.close();
        fs.close();
    }
}
```

This code creates a new file in HDFS at the specified path and writes the string "Hello, HDFS!" to it. The `fs.defaultFS` property specifies the HDFS URI, and the `FileSystem` object is used to interact with the file system. The `BufferedWriter` is used to write data to the file.

Design of HDFS

HDFS is designed to store very large data sets reliably and to stream those data sets at high bandwidth to user applications. It is built to run on commodity hardware and is highly fault-tolerant. The architecture of HDFS is based on a master/slave model, where the master is the NameNode and the slaves are the

DataNodes.

The NameNode manages the file system namespace and regulates access to files by clients. It also executes file system operations such as renaming, closing, and opening files and directories. The DataNodes are responsible for serving read and write requests from the file system's clients, and they also perform block creation, deletion, and replication upon instruction from the NameNode.

HDFS stores files as blocks, and the block size is configurable. Each block is stored on multiple DataNodes, and the number of replicas is also configurable. The NameNode determines the mapping of blocks to DataNodes, and it periodically receives a report from each DataNode about the blocks it is storing.

HDFS is designed to be accessed by a small number of very large files, rather than a large number of small files. It is optimized for streaming data access, and it is not suitable for low-latency data access. HDFS also provides interfaces for applications to move themselves closer to where the data is located, to reduce the amount of data that must be transferred over the network.

HDFS concepts

HDFS (Hadoop Distributed File System) is a distributed file system designed to run on commodity hardware. It is highly fault-tolerant and is designed to be deployed on low-cost hardware. HDFS provides high throughput access to application data and is suitable for applications that have large data sets.

Some of the key concepts of HDFS include:

- **NameNode and DataNode:** HDFS has a master/slave architecture. The NameNode is the master server that manages the file system namespace and regulates access to files by clients. The DataNodes are the slave servers that manage the storage attached to the nodes that they run on.
- **Block size:** HDFS stores large files as a sequence of blocks. The default block size is 64MB, but it can be configured by the user.
- **Replication:** HDFS replicates each block of data on multiple DataNodes to ensure high availability and fault tolerance. The default replication factor is 3, but it can be configured by the user.
- **Rack awareness:** HDFS is designed to be aware of the network topology of the cluster. It tries to place replicas of data blocks on different racks to improve data reliability and availability.
- **Data locality:** HDFS tries to schedule tasks on the same node where the data is stored, or as close as possible, to reduce network traffic and improve performance.
- **Scalability:** HDFS is designed to scale to thousands of nodes and petabytes of data.

Benefits of HDFS

HDFS (Hadoop Distributed File System) is a distributed file system designed to run on commodity hardware. It has many benefits, including:

1. **Scalability:** HDFS can easily scale to handle petabytes of data by simply adding more nodes to the cluster.
2. **Fault tolerance:** HDFS is designed to be highly fault-tolerant, with data automatically replicated across multiple nodes to ensure data availability in the event of node failure.
3. **Cost-effective:** HDFS runs on commodity hardware, making it a cost-effective solution for storing and processing large amounts of data.
4. **Data locality:** HDFS takes advantage of data locality by moving computation to the data, rather than moving data to the computation. This can significantly improve performance for data-intensive applications.
5. **High throughput:** HDFS is optimized for high-throughput, batch processing of large data sets, making it well-suited for big data analytics and machine learning applications.

Challenges of HDFS

Hadoop Distributed File System (HDFS) is a distributed file system designed to run on commodity hardware. It has many advantages, but there are also several challenges associated with it. Some of the challenges of HDFS include:

1. **Scalability:** As the amount of data stored in HDFS increases, it becomes more challenging to scale the system to accommodate the growth. This can result in performance degradation and increased costs.
2. **Data Integrity:** Ensuring the integrity of data stored in HDFS can be challenging, especially in the face of hardware failures or network disruptions. HDFS employs various mechanisms to ensure data integrity, but these can add complexity to the system.
3. **Data Accessibility:** HDFS is designed to store large amounts of data, but accessing that data can be challenging. The system is optimized for batch processing, which can make it difficult to perform real-time data analysis or interactive queries.
4. **Data Management:** Managing the data stored in HDFS can be challenging, especially as the amount of data grows. This can include tasks such as balancing data across nodes, ensuring data availability, and performing backups.

These are just some of the challenges associated with HDFS. Despite these challenges, HDFS remains a popular and powerful tool for storing and processing large amounts of data.

File sizes in HDFS

Here is an example of how to get the file sizes in HDFS using Python and the `hdfs` library:

```
from hdfs import InsecureClient

# Connect to HDFS
client = InsecureClient('http://namenode:50070', user='hdfs')

# Get the file status
status = client.status('/path/to/file')

# Get the file size
file_size = status['length']

# Print the file size
print(f'File size: {file_size} bytes')
```

Block Sizes in HDFS

In Hadoop Distributed File System (HDFS), the default block size is 128 MB. This can be changed by modifying the `dfs.blocksize` parameter in the `hdfs-site.xml` configuration file. Here is an example of how to change the block size to 256 MB:

```
<configuration>
  <property>
    <name>dfs.blocksize</name>
    <value>268435456</value>
    <description>Block size in bytes.</description>
  </property>
</configuration>
```

After changing the block size, the Hadoop cluster needs to be restarted for the changes to take effect. It is important to note that changing the block size will only affect new files that are added to HDFS. Existing files will still have the old block size.

Block Abstraction in HDFS

In Hadoop Distributed File System (HDFS), a file is split into one or more blocks and these blocks are stored in a set of DataNodes. The NameNode executes file system namespace operations like opening, closing, and renaming files and directories. It also determines the mapping of blocks to DataNodes. The DataNodes are responsible for serving read and write requests from the file system's clients. The DataNodes also perform block creation, deletion, and replication upon instruction from the NameNode.

Here is an example of how block abstraction can be implemented in HDFS using Java:

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.BlockLocation;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;

import java.io.IOException;

public class HDFSBlockAbstraction {
    public static void main(String[] args) throws IOException {
        Configuration conf = new Configuration();
        conf.set("fs.defaultFS", "hdfs://localhost:9000");
        FileSystem fs = FileSystem.get(conf);
        Path filePath = new Path("/path/to/file");
        BlockLocation[] blockLocations = fs.getFileBlockLocations(filePath, 0, fs.getFileStatus().getLength());
        for (BlockLocation blockLocation : blockLocations) {
            System.out.println("Block offset: " + blockLocation.getOffset());
            System.out.println("Block length: " + blockLocation.getLength());
            System.out.println("Hosts: ");
            for (String host : blockLocation.getHosts()) {
                System.out.println(host);
            }
        }
    }
}
```

This code demonstrates how to retrieve the block locations of a file in HDFS using the `getFileBlockLocations` method of the `FileSystem` class. The `BlockLocation` objects returned by this method contain information about the offset, length, and hosts of each block. This information can be used to perform data locality optimizations when processing the file.

Data replication in HDFS

Hadoop Distributed File System (HDFS) is a distributed file system designed to store large data sets reliably and to stream those data sets at high bandwidth to user applications. One of the key features of HDFS is its ability to replicate data across multiple nodes in a cluster to ensure data availability and durability.

Here is an example of how data replication can be configured in HDFS using the `hdfs-site.xml` configuration file:

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>3</value>
```

```

    <description>Default block replication.
    The actual number of replications can be specified when the file is created.
    The default is used if replication is not specified in create time.
    </description>
  </property>
</configuration>

```

In this example, the `dfs.replication` property is set to 3, which means that HDFS will create 3 replicas of each data block by default. This value can be adjusted to meet the specific needs of the cluster and the data being stored.

How does HDFS store

Hadoop Distributed File System (HDFS) is a distributed file system designed to store large data sets across multiple machines. HDFS stores data by dividing it into blocks and distributing these blocks across the nodes in the cluster. Each block is typically 128 MB in size and is replicated multiple times (usually 3) across different nodes for fault tolerance. When a file is written to HDFS, the NameNode, which is the master node in the HDFS cluster, determines the location of the data blocks and coordinates the write operation with the DataNodes, which are the worker nodes that store the data blocks. When a file is read from HDFS, the NameNode provides the location of the data blocks to the client, which then reads the data directly from the DataNodes.

Here is an example of how HDFS stores data in Java:

```

Configuration conf = new Configuration();
conf.set("fs.defaultFS", "hdfs://namenode:8020");
FileSystem fs = FileSystem.get(conf);

Path filePath = new Path("/path/to/file");
FSDataOutputStream outputStream = fs.create(filePath);

outputStream.write("Data to be written to HDFS".getBytes());
outputStream.close();

```

This code creates a `Configuration` object and sets the HDFS URI. It then creates a `FileSystem` object and uses it to create a file in HDFS. The `FSDataOutputStream` object is used to write data to the file, and the stream is closed when the write operation is complete. This code assumes that the HDFS cluster is running and that the client has the necessary permissions to write to the specified path in HDFS.

Read operations in HDFS

Here is an example of how to perform read operations in HDFS using the Hadoop `FileSystem` API:

```

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import java.io.InputStream;
import java.net.URI;

public class HDFSRead {
    public static void main(String[] args) throws Exception {
        String uri = args[0];
        Configuration conf = new Configuration();
        FileSystem fs = FileSystem.get(URI.create(uri), conf);
        InputStream in = null;
        try {
            in = fs.open(new Path(uri));
            // Read data from the input stream
        } finally {
            in.close();
        }
    }
}

```

This code reads data from a file in HDFS by opening an input stream to the file and reading data from the stream. The input stream is closed in the `finally` block to ensure that resources are released properly.

Write Operations in HDFS

Hadoop Distributed File System (HDFS) is a distributed file system designed to store large data sets reliably and to stream those data sets at high bandwidth to user applications. Here are some key points to remember about write operations in HDFS:

1. **Data Replication:** HDFS replicates data blocks for fault tolerance. The default replication factor is 3, meaning that HDFS stores three copies of each data block on different nodes.
2. **Data Pipelining:** When a client writes data to HDFS, the data is first written to the local disk of the client machine. Then, the data is sent to the first DataNode in the pipeline. The first DataNode stores the data and forwards it to the second DataNode in the pipeline, and so on.
3. **Data Integrity:** HDFS uses checksums to ensure data integrity. When a client writes data to HDFS, it computes a checksum for each data block and sends the checksum to the DataNode along with the data. The DataNode verifies the checksum before storing the data.
4. **Write-once-read-many:** HDFS follows the write-once-read-many model. Once a file is created, written, and closed, it cannot be modified.

However, it can be read any number of times.

5. **Atomicity:** HDFS supports atomicity for write operations. This means that a write operation is either completed successfully or not at all. If a write operation fails, the file system state is unchanged.
6. **Data Locality:** HDFS tries to place data blocks on the same node or rack as the client writing the data. This improves data locality and reduces network traffic.
7. **Data Coherency:** HDFS provides data coherency by ensuring that once a write operation is completed and the file is closed, all subsequent reads will see the new data.

Java interfaces to HDFS

Hadoop Distributed File System (HDFS) is a distributed file system designed to run on commodity hardware. It has many similarities with existing distributed file systems. However, the differences from other distributed file systems are significant. HDFS is highly fault-tolerant and is designed to be deployed on low-cost hardware. HDFS provides high throughput access to application data and is suitable for applications that have large data sets.

Here is an example of how to use the Java interface to interact with HDFS:

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;

import java.io.IOException;

public class HdfsExample {
    public static void main(String[] args) throws IOException {
        Configuration conf = new Configuration();
        conf.set("fs.defaultFS", "hdfs://localhost:9000");
        FileSystem fs = FileSystem.get(conf);
        Path path = new Path("/user/hadoop/test.txt");
        if (fs.exists(path)) {
            System.out.println("File exists");
        } else {
            System.out.println("File does not exist");
        }
        fs.close();
    }
}
```

This code snippet shows how to create a `Configuration` object and set the Hadoop file system to HDFS. Then, it creates a `FileSystem` object and uses it to check if a file exists in HDFS. Finally, it closes the `FileSystem` object. This

is just one example of how to use the Java interface to interact with HDFS. There are many other operations that can be performed, such as reading and writing files, creating and deleting directories, and more.

Command Line Interface to HDFS

Here is an example of how to use the command line interface to interact with HDFS:

```
# List the contents of the root directory in HDFS
hdfs dfs -ls /

# Create a new directory in HDFS
hdfs dfs -mkdir /new_directory

# Copy a file from the local file system to HDFS
hdfs dfs -put local_file.txt /new_directory

# View the contents of a file in HDFS
hdfs dfs -cat /new_directory/local_file.txt

# Delete a file in HDFS
hdfs dfs -rm /new_directory/local_file.txt

# Delete a directory in HDFS
hdfs dfs -rmdir /new_directory
```

Hadoop file system interfaces

The Hadoop Distributed File System (HDFS) is a distributed file system designed to run on commodity hardware. It has many similarities with existing distributed file systems. However, the differences from other distributed file systems are significant. HDFS is highly fault-tolerant and is designed to be deployed on low-cost hardware. HDFS provides high throughput access to application data and is suitable for applications that have large data sets.

Here is an example of how to use the Hadoop file system interfaces in Java:

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;

import java.io.IOException;

public class HadoopFileSystemExample {
    public static void main(String[] args) throws IOException {
        Configuration conf = new Configuration();
        conf.set("fs.defaultFS", "hdfs://localhost:9000");
```

```

        FileSystem fs = FileSystem.get(conf);
        Path path = new Path("/user/hadoop/test.txt");
        if (fs.exists(path)) {
            System.out.println("File exists");
        } else {
            System.out.println("File does not exist");
        }
        fs.close();
    }
}

```

This code creates a `Configuration` object and sets the Hadoop file system to be the default file system. It then creates a `FileSystem` object and uses it to check if a file exists in HDFS. Finally, it closes the `FileSystem` object. This is just one example of how to use the Hadoop file system interfaces. There are many other methods available for interacting with HDFS.

Data flow in HDFS

Here is an example of how data flows in HDFS when a file is being written:

1. The client opens a file for writing by calling the `create()` method on the `DistributedFileSystem` object.
2. The `DistributedFileSystem` object communicates with the `NameNode` to create a new file in the file system's namespace.
3. The `NameNode` performs various checks to ensure that the file can be created, such as checking if the file already exists and if the client has the necessary permissions to create the file.
4. If the checks pass, the `NameNode` creates the file and returns a `FSDDataOutputStream` object to the client.
5. The client writes data to the `FSDDataOutputStream` object, which is then split into packets and sent to the `DataNode` that is responsible for storing the first block of the file.
6. The `DataNode` stores the data and sends an acknowledgment to the client.
7. The client continues to write data to the `FSDDataOutputStream` object, which is then sent to the `DataNode` responsible for storing the next block of the file.
8. This process continues until the client has finished writing the file.
9. When the client is finished writing the file, it calls the `close()` method on the `FSDDataOutputStream` object.
10. The `DataNode` responsible for storing the last block of the file notifies the `NameNode` that the file has been closed.
11. The `NameNode` updates the file system's metadata to reflect that the file has been closed.

This is a high-level overview of how data flows in HDFS when a file is being written. There are many more details involved in the process, such as data

replication and error handling, but this should give you a general idea of how it works.

Data Ingest with Flume and Sqoop in HDFS

Here is an example of how you can ingest data into HDFS using Flume and Sqoop:

1. **Flume:** Flume is a distributed, reliable, and available service for efficiently collecting, aggregating, and moving large amounts of log data. Here is an example of a Flume configuration file that can be used to ingest data into HDFS:

```
# Define a memory channel called ch1 on agent1
agent1.channels.ch1.type = memory

# Define an HDFS sink that writes to HDFS
agent1.sinks.sink1.type = hdfs
agent1.sinks.sink1.hdfs.path = /path/to/hdfs
agent1.sinks.sink1.hdfs.fileType = DataStream
agent1.sinks.sink1.channel = ch1

# Define a source that reads data from an external source
agent1.sources.source1.type = netcat
agent1.sources.source1.bind = localhost
agent1.sources.source1.port = 12345
agent1.sources.source1.channels = ch1
```

2. **Sqoop:** Sqoop is a tool designed for efficiently transferring bulk data between Apache Hadoop and structured datastores such as relational databases. Here is an example of a Sqoop command that can be used to import data from a relational database into HDFS:

```
sqoop import \
--connect jdbc:mysql://database.example.com/db \
--username user \
--password pass \
--table tablename \
--target-dir /path/to/hdfs \
--num-mappers 4
```

This command will import data from the `tablename` table in the `db` database on the `database.example.com` server into the `/path/to/hdfs` directory in HDFS using 4 mappers. You can adjust the number of mappers to optimize the performance of the data transfer.

Hadoop archives in HDFS

Hadoop archives (HAR files) are a way to reduce the number of files in HDFS, by combining smaller files into larger archives. This can improve the performance of HDFS by reducing the load on the NameNode.

Here is an example of how to create a Hadoop archive using the `hadoop archive` command:

```
hadoop archive -archiveName myArchive.har -p /input /output
```

This command will create a Hadoop archive named `myArchive.har` from the files in the `/input` directory and store the archive in the `/output` directory.

You can then access the files in the archive using the `har://` URI scheme. For example, to access a file named `file.txt` in the archive, you would use the following URI: `har:///output/myArchive.har/file.txt`.

Hadoop I/O

Hadoop provides its own implementations of several standard Java I/O classes, which are optimized for use within the Hadoop environment. Here is an example of how to use Hadoop's `SequenceFile` class to write key-value pairs to a file:

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.SequenceFile;
import org.apache.hadoop.io.Text;

Configuration conf = new Configuration();
FileSystem fs = FileSystem.get(conf);
Path path = new Path("/tmp/sequencefile");

IntWritable key = new IntWritable();
Text value = new Text();
SequenceFile.Writer writer = SequenceFile.createWriter(fs, conf, path, key.getClass(), value.getClass());

for (int i = 0; i < 100; i++) {
    key.set(i);
    value.set("Value" + i);
    writer.append(key, value);
}

writer.close();
```

This code creates a new `SequenceFile.Writer` object, which is used to write key-value pairs to a file located at `/tmp/sequencefile`. The key and value classes are specified as `IntWritable` and `Text`, respectively. The `for` loop writes

100 key-value pairs to the file, where the key is an integer and the value is a string. Finally, the `writer` is closed to flush all data to the file.

Compression in Hadoop IO

Hadoop provides support for various compression codecs and formats. Here is an example of how to compress and decompress data using the `GzipCodec` in Hadoop:

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IOUtils;
import org.apache.hadoop.io.compress.CompressionCodec;
import org.apache.hadoop.io.compress.CompressionCodecFactory;
import org.apache.hadoop.io.compress.GzipCodec;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.InputStream;
import java.io.OutputStream;

public class CompressionExample {
    public static void main(String[] args) throws Exception {
        String inputPath = args[0];
        String outputPath = args[1];

        Configuration conf = new Configuration();
        CompressionCodecFactory factory = new CompressionCodecFactory(conf);
        CompressionCodec codec = factory.getCodec(new Path(outputPath));
        if (codec == null) {
            System.err.println("No codec found for " + outputPath);
            System.exit(1);
        }

        InputStream in = null;
        OutputStream out = null;
        try {
            in = new FileInputStream(inputPath);
            out = codec.createOutputStream(new FileOutputStream(outputPath));
            IOUtils.copyBytes(in, out, conf);
        } finally {
            IOUtils.closeStream(in);
            IOUtils.closeStream(out);
        }
    }
}
```

This code reads data from the input path, compresses it using the `GzipCodec`, and writes the compressed data to the output path. To decompress the data, you can use the `createInputStream` method of the codec instead of the `createOutputStream` method.

Serialization in Hadoop IO

Serialization is the process of converting data structures or objects into a binary or textual format that can be stored or transmitted and later deserialized back into its original form. In Hadoop, serialization is used to transfer data between nodes and to write data to disk.

Here is an example of how to implement a custom `Writable` class in Hadoop for serialization:

```
import org.apache.hadoop.io.Writable;
import java.io.DataInput;
import java.io.DataOutput;
import java.io.IOException;

public class MyWritable implements Writable {
    private int myInt;
    private String myString;

    public MyWritable() {}

    public MyWritable(int myInt, String myString) {
        this.myInt = myInt;
        this.myString = myString;
    }

    @Override
    public void write(DataOutput out) throws IOException {
        out.writeInt(myInt);
        out.writeUTF(myString);
    }

    @Override
    public void readFields(DataInput in) throws IOException {
        myInt = in.readInt();
        myString = in.readUTF();
    }
}
```

This class implements the `Writable` interface and overrides the `write` and `readFields` methods to define how the data should be serialized and deserialized. The `write` method writes the data to the `DataOutput` stream, and the

readFields method reads the data from the DataInput stream.

Avro and file based data structures in Hadoop io

Avro is a data serialization system that is used to encode data into a compact binary format. It is commonly used in Hadoop to store and process large amounts of data. Avro data is stored in a file-based data structure, which means that the data is stored in files on the Hadoop Distributed File System (HDFS).

Here is an example of how to write Avro data to a file in Hadoop:

```
import org.apache.avro.file.DataFileWriter;
import org.apache.avro.io.DatumWriter;
import org.apache.avro.specific.SpecificDatumWriter;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;

// Create a configuration object
Configuration conf = new Configuration();

// Set the Hadoop file system
conf.set("fs.defaultFS", "hdfs://localhost:9000");

// Create a Hadoop file system object
FileSystem fs = FileSystem.get(conf);

// Create a path for the Avro file
Path path = new Path("/user/hadoop/avrodata.avro");

// Create an output stream to write to the Avro file
FSDataOutputStream out = fs.create(path);

// Create a datum writer to write the Avro data
DatumWriter<MyAvroRecord> writer = new SpecificDatumWriter<>(MyAvroRecord.class);

// Create a data file writer to write the Avro data to the file
DataFileWriter<MyAvroRecord> dataFileWriter = new DataFileWriter<>(writer);

// Open the data file writer
dataFileWriter.create(MyAvroRecord.SCHEMA$, out);

// Write the Avro data to the file
dataFileWriter.append(myAvroRecord);

// Close the data file writer
dataFileWriter.close();
```

Hadoop Environment

Here is an example of how to set up a Hadoop environment in a Linux system:

1. Install Java: Hadoop requires Java to run. You can install the latest version of Java by running the following command:

```
sudo apt-get update
sudo apt-get install default-jdk
```

2. Download Hadoop: You can download the latest version of Hadoop from the Apache Hadoop website. Once downloaded, extract the files to a directory of your choice.
3. Set environment variables: You need to set the `JAVA_HOME` and `HADOOP_HOME` environment variables. You can do this by adding the following lines to your `.bashrc` file:

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=/path/to/hadoop/directory
export PATH=$PATH:$HADOOP_HOME/bin
```

Remember to replace `/path/to/hadoop/directory` with the actual path to your Hadoop directory.

4. Configure Hadoop: Hadoop requires some configuration before it can be used. You can find the configuration files in the `etc/hadoop` directory within your Hadoop installation. The most important files to configure are `core-site.xml`, `hdfs-site.xml`, and `mapred-site.xml`.
5. Start Hadoop: Once you have completed the above steps, you can start Hadoop by running the following command from the Hadoop directory:

```
./sbin/start-all.sh
```

Setting up a Hadoop cluster in Hadoop Environment

Here is an example of how to set up a Hadoop cluster in a Hadoop environment:

1. Install Hadoop on all the machines that will be part of the cluster.
2. Configure the `core-site.xml`, `hdfs-site.xml`, and `mapred-site.xml` files on all the machines.
3. On the master machine, format the Hadoop Distributed File System (HDFS) by running the command `hadoop namenode -format`.
4. Start the Hadoop daemons on all the machines by running the command `start-all.sh`.
5. Verify that the cluster is running by checking the web interface of the NameNode and the JobTracker.

This is just a basic example of how to set up a Hadoop cluster. There are many other configurations and settings that can be adjusted to optimize the perfor-

mance of the cluster. It is recommended to consult the Hadoop documentation for more detailed information.

Cluster Specification in Hadoop Environment

Here is an example of a cluster specification in a Hadoop environment:

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://namenode:8020</value>
  </property>
  <property>
    <name>yarn.resourcemanager.hostname</name>
    <value>resourcemanager</value>
  </property>
  <property>
    <name>yarn.nodemanager.resource.memory-mb</name>
    <value>4096</value>
  </property>
  <property>
    <name>yarn.scheduler.maximum-allocation-mb</name>
    <value>2048</value>
  </property>
  <property>
    <name>yarn.scheduler.minimum-allocation-mb</name>
    <value>1024</value>
  </property>
  <property>
    <name>yarn.nodemanager.vmem-check-enabled</name>
    <value>false</value>
  </property>
</configuration>
```

This is an example of a configuration file for a Hadoop cluster. It specifies the default file system, the hostname of the resource manager, the amount of memory available to the node manager, and the maximum and minimum memory allocation for the scheduler. It also disables virtual memory checking by the node manager.

Cluster setup and installation in Hadoop Environment

Here is an example of how to set up and install a Hadoop cluster:

1. Install Java on all the nodes in the cluster.
2. Download and install Hadoop on all the nodes.
3. Configure the `core-site.xml`, `hdfs-site.xml`, and `mapred-site.xml` files on all the nodes.

4. Set up password-less SSH between all the nodes.
5. Format the Hadoop file system on the NameNode.
6. Start the Hadoop daemons on all the nodes: NameNode, DataNode, ResourceManager, and NodeManager.
7. Verify that the cluster is up and running by checking the web interface or running a test job.

This is just one example of how to set up and install a Hadoop cluster. There are many different ways to do it, and the specific steps may vary depending on the specific needs and requirements of the cluster. It is important to carefully plan and configure the cluster to ensure optimal performance and reliability.

Hadoop Configuration in Hadoop Environment

To configure Hadoop in a Hadoop environment, you need to edit the configuration files located in the `$HADOOP_HOME/etc/hadoop` directory. Here are the steps to configure Hadoop:

1. Edit the `core-site.xml` file to set the Hadoop configuration properties such as the default file system name and the default block size.

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
  <property>
    <name>io.file.buffer.size</name>
    <value>131072</value>
  </property>
</configuration>
```

2. Edit the `hdfs-site.xml` file to set the HDFS configuration properties such as the replication factor and the name directory.

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///usr/local/hadoop/hadoop_data/hdfs/namenode</value>
  </property>
</configuration>
```

3. Edit the `mapred-site.xml` file to set the MapReduce configuration properties such as the framework name and the job tracker address.

```

<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
  <property>
    <name>mapreduce.jobtracker.address</name>
    <value>localhost:54311</value>
  </property>
</configuration>

```

4. Edit the `yarn-site.xml` file to set the YARN configuration properties such as the resource manager address and the node manager address.

```

<configuration>
  <property>
    <name>yarn.resourcemanager.hostname</name>
    <value>localhost</value>
  </property>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>

```

After editing the configuration files, you need to format the HDFS file system and start the Hadoop daemons.

```

$HADOOP_HOME/bin/hdfs namenode -format
$HADOOP_HOME/sbin/start-dfs.sh
$HADOOP_HOME/sbin/start-yarn.sh

```

Security in Hadoop Environment

Hadoop provides several security features to protect data and maintain the integrity of the cluster. Here is an example of how to configure security in a Hadoop environment:

1. Enable Kerberos authentication: Kerberos is a network authentication protocol that can be used to secure communication between Hadoop services. To enable Kerberos, you need to configure the `core-site.xml` and `hdfs-site.xml` files with the appropriate settings.

```

<property>
  <name>hadoop.security.authentication</name>
  <value>kerberos</value>
</property>

```

2. Set up Hadoop Access Control: Hadoop provides several mechanisms for

controlling access to data, including file and directory permissions, Access Control Lists (ACLs), and storage policies. These can be configured using the `hdfs dfs` command or the Hadoop web UI.

3. Configure data encryption: Hadoop supports encryption of data at rest and in transit. To enable encryption, you need to configure the `hdfs-site.xml` file with the appropriate settings and set up a Key Management Server (KMS).

```
<property>
  <name>dfs.encrypt.data.transfer</name>
  <value>true</value>
</property>
```

These are just a few examples of how to configure security in a Hadoop environment. It is important to carefully plan and implement security measures to protect your data and maintain the integrity of your cluster.

Administering Hadoop in Hadoop Environment

Here is an example of how to administer Hadoop in a Hadoop environment:

```
# Start the Hadoop daemons
$HADOOP_HOME/sbin/start-dfs.sh
$HADOOP_HOME/sbin/start-yarn.sh

# Check the status of the Hadoop daemons
jps

# Stop the Hadoop daemons
$HADOOP_HOME/sbin/stop-yarn.sh
$HADOOP_HOME/sbin/stop-dfs.sh
```

HDFS monitoring & maintenance in Hadoop Environment

HDFS monitoring and maintenance is an important aspect of managing a Hadoop environment. Here is an example of code that can be used to monitor and maintain HDFS:

```
from subprocess import check_output

# Function to get the HDFS disk usage
def get_hdfs_disk_usage():
    cmd = "hdfs dfs -du -h /"
    output = check_output(cmd, shell=True)
    return output

# Function to get the HDFS report
def get_hdfs_report():
```



```

    cmd = "hdfs dfsadmin -report"
    output = check_output(cmd, shell=True)
    return output

# Function to get the HDFS file system check
def get_hdfs_fsck():
    cmd = "hdfs fsck /"
    output = check_output(cmd, shell=True)
    return output

# Function to get the HDFS balancer status
def get_hdfs_balancer_status():
    cmd = "hdfs balancer -status"
    output = check_output(cmd, shell=True)
    return output

# Function to run the HDFS balancer
def run_hdfs_balancer():
    cmd = "hdfs balancer"
    output = check_output(cmd, shell=True)
    return output

# Example usage
disk_usage = get_hdfs_disk_usage()
report = get_hdfs_report()
fsck = get_hdfs_fsck()
balancer_status = get_hdfs_balancer_status()
balancer_output = run_hdfs_balancer()

print(disk_usage)
print(report)
print(fsck)
print(balancer_status)
print(balancer_output)

```

This code provides functions to get the HDFS disk usage, report, file system check, balancer status, and to run the HDFS balancer. These functions can be used to monitor and maintain the HDFS in a Hadoop environment.

Hadoop benchmarks in Hadoop Environment

Here is an example of a Hadoop benchmark code that can be used in a Hadoop environment:

```

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;

```

```

import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable>{

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context
            ) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {
        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values,
            Context context
            ) throws IOException, InterruptedException {

            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
    }
}

```

```

        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

This code is an example of a WordCount benchmark, which is a common benchmark used to measure the performance of a Hadoop cluster. It counts the number of occurrences of each word in a given input file and outputs the results.

Hadoop in the cloud in Hadoop Environment

Here is an example of code that can be used to set up Hadoop in the cloud in a Hadoop environment:

```

from pyspark import SparkConf, SparkContext
from pyspark.sql import SparkSession

# Set up the Spark configuration
conf = SparkConf().setAppName("Hadoop in the cloud").setMaster("local[*]")

# Create the Spark context
sc = SparkContext.getOrCreate(conf)

# Create the Spark session
spark = SparkSession.builder.appName("Hadoop in the cloud").getOrCreate()

# Load data from HDFS
data = sc.textFile("hdfs://namenode:8020/path/to/data")

# Perform data processing
processed_data = data.map(lambda x: x.split(",")).filter(lambda x: x[0] == "some_value")

# Save the processed data back to HDFS
processed_data.saveAsTextFile("hdfs://namenode:8020/path/to/output")

```

Unit 3 - Hadoop Eco System and YARN , no SQL databases , MongoDB , Spark , Scala

```

// Hadoop Eco System and YARN
import org.apache.hadoop.yarn.api.records.ApplicationId
import org.apache.hadoop.yarn.client.api.YarnClient

```

```

import org.apache.hadoop.yarn.conf.YarnConfiguration

val conf = new YarnConfiguration()
val yarnClient = YarnClient.createYarnClient()
yarnClient.init(conf)
yarnClient.start()

val app = yarnClient.createApplication()
val appId: ApplicationId = app.getNewApplicationResponse().getApplicationId()

// no SQL databases
import com.mongodb.casbah.Imports._

val mongoClient = MongoClient("localhost", 27017)
val db = mongoClient("mydb")
val coll = db("test")

val doc = MongoDBObject("name" -> "MongoDB", "type" -> "database", "count" -> 1, "info" -> M
coll.insert(doc)

// Spark
import org.apache.spark.SparkConf
import org.apache.spark.SparkContext

val conf = new SparkConf().setAppName("MyApp").setMaster("local")
val sc = new SparkContext(conf)

val data = Array(1, 2, 3, 4, 5)
val distData = sc.parallelize(data)

// Scala
val x = 1
val y = 2
val z = x + y
println(z)

```

Hadoop Eco System and YARN

Hadoop is an open-source software framework for storing and processing large datasets. The Hadoop ecosystem consists of several components, including Hadoop Distributed File System (HDFS), MapReduce, and Yet Another Resource Negotiator (YARN).

YARN is the resource management layer of Hadoop. It is responsible for managing and allocating resources to applications running on the Hadoop cluster. YARN separates the resource management and scheduling functions from the

data processing component, allowing for more efficient and scalable processing of data.

Here is an example of how YARN works in the Hadoop ecosystem:

```
from hadoop.yarn import api

# Create a YARN client
client = api.YarnClient()

# Submit a new application to the YARN cluster
app = client.submit_application(
    name="my-app",
    queue="default",
    memory=1024,
    vcores=1,
    command="my-command"
)

# Monitor the application's progress
while app.state not in ["FINISHED", "FAILED", "KILLED"]:
    app = client.get_application(app.id)
    print("Application state:", app.state)
    time.sleep(1)

# Get the final application report
report = client.get_application_report(app.id)
print("Application report:", report)
```

This code creates a YARN client, submits a new application to the YARN cluster, monitors the application's progress, and retrieves the final application report. This is just one example of how YARN can be used in the Hadoop ecosystem to manage resources and run applications.

Hadoop Ecosystem Components

The Hadoop ecosystem is a framework and set of tools for processing large amounts of data. Here are some of the key components of the Hadoop ecosystem:

1. **Hadoop Distributed File System (HDFS):** A distributed file system that provides high-throughput access to application data.
2. **MapReduce:** A programming model for processing large data sets with a parallel, distributed algorithm on a cluster.
3. **YARN:** A resource management platform responsible for managing compute resources in clusters and using them for scheduling of users' applications.

4. **HBase:** A non-relational, distributed database that runs on top of HDFS.
5. **Hive:** A data warehousing and SQL-like query language for Hadoop.
6. **Pig:** A high-level platform for creating MapReduce programs used with Hadoop.
7. **Sqoop:** A tool for transferring bulk data between Apache Hadoop and structured data stores such as relational databases.
8. **Flume:** A service for collecting, aggregating, and moving large amounts of log data.
9. **Oozie:** A workflow scheduler system to manage Apache Hadoop jobs.
10. **ZooKeeper:** A centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services.

These are some of the key components of the Hadoop ecosystem, each serving a specific purpose in processing large amounts of data.

Schedulers in Hadoop ecosystem

In the Hadoop ecosystem, there are several schedulers available to manage the allocation of resources to different jobs. Some of the most commonly used schedulers are:

1. **FIFO Scheduler:** This is the simplest scheduler, where jobs are executed in the order they are submitted to the cluster. It is suitable for small clusters with a low volume of jobs.
2. **Fair Scheduler:** This scheduler allocates resources to jobs in a way that ensures that all jobs get, on average, an equal share of resources over time. It is suitable for large clusters with a high volume of jobs.
3. **Capacity Scheduler:** This scheduler is designed to allow multiple tenants to share a large cluster while ensuring that each tenant receives a guaranteed minimum share of the resources. It is suitable for multi-tenant clusters.

Each of these schedulers has its own set of configuration parameters that can be tuned to meet the specific needs of the cluster and the jobs running on it. It is important to choose the right scheduler and configure it properly to ensure efficient utilization of the cluster resources.

Fair and Capacity in Hadoop Ecosystem

- Hadoop is a batch processing ecosystem that cannot analyze data on-the-fly. It uses schedulers to schedule tasks in a Hadoop cluster when it receives requests from different clients .

- There are mainly 3 types of Schedulers in Hadoop: FIFO (First In First Out) Scheduler, Capacity Scheduler, and Fair Scheduler .
- Fair Scheduler allows YARN applications to justly share resources in large Hadoop clusters. With this scheduler, there is no need for reserving a set amount of capacity because it will dynamically balance resources between all running applications .
- Fair scheduling is a method of assigning resources to jobs such that all jobs get, on average, an equal share of resources over time. When there is a single job running, that job uses the entire cluster. When other jobs are submitted, tasks slots that free up are assigned to the new jobs, so that each job gets roughly the same amount of CPU time .
- The Fair Scheduler is very much similar to that of the capacity scheduler. The priority of the job is kept in consideration. With the help of Fair Scheduler, the YARN applications can share the resources in the large Hadoop Cluster and these resources are maintained dynamically so no need for prior capacity .

Hadoop 2.0 New Features - NameNode high availability

- Hadoop 2.0 introduced the High Availability feature to solve the Single Point of Failure (SPOF) problem in older versions of Hadoop .
- The Hadoop HDFS follows a master-slave architecture where the NameNode is the master node and maintains the filesystem tree .
- Hadoop 2.0 overcomes the SPOF shortcoming by providing support for multiple NameNodes .
- It introduces an extra NameNode (Passive Standby NameNode) to the Hadoop Architecture, which is configured for automatic failover .
- The main motive of the Hadoop 2.0 High Availability project is to render availability to large data applications 24/7 through the deployment of 2 Hadoop NameNodes .
- This eliminates the NameNode as a potential single point of failure (SPOF) in an HDFS cluster .

HDFS Federation in Hadoop Ecosystem

- HDFS Federation is a feature introduced in Hadoop 2 that enhances the existing HDFS architecture.
- It overcomes HDFS architecture limitations by adding support for multiple NameNodes/namespaces to HDFS. This allows the use of more than one NameNode/namespace.
- HDFS Federation overcomes the isolation, scalability, and performance limitations of the prior HDFS architecture.
- The HDFS Federation architecture has a collection of Namespace volumes, which are self-contained management units. On deleting a NameNode or namespace, the corresponding block pool present in the DataNodes also gets deleted. On upgrading the cluster, each namespace volume gets

upgraded as a unit.

- HDFS has two main layers: Namespace and Block Storage Service. Namespace consists of directories, files, and blocks and supports all namespace-related file system operations such as creating, deleting, modifying, and listing files and directories. Block Storage Service has two parts: Block Management (performed in the NameNode) and Block Storage (performed in the DataNodes).

MRv2 in Hadoop ecosystem

- MRv2, also known as Hadoop 2, is a version of Hadoop where the resource management and scheduling tasks are separated from MapReduce by YARN (Yet Another Resource Negotiator) .
- In Hadoop 2, the resource management and scheduling layer lies beneath the MapReduce layer .
- MRv2 is an application framework that runs within YARN .
- In Hadoop 1, all DataNodes are dedicated to Map and Reduce tasks and cannot be used for other processing .
- In Hadoop 1, the cluster's capacity is measured in MapReduce slots .
- In Hadoop version 1, MapReduce was responsible for both processing and cluster resource management .
- In Apache Hadoop version 2, cluster resource management has been moved from MapReduce into YARN, thus enabling other application engines to utilize YARN and Hadoop, while also improving the performance of MapReduce .
- Hadoop 2 has undergone a complete change in terms of architecture and components compared to Hadoop 1 .

YARN

- YARN stands for Yet Another Resource Negotiator.
- It is a large-scale, distributed operating system for big data applications.
- YARN is responsible for managing resources and scheduling tasks in a Hadoop cluster.
- It was introduced in Hadoop 2.0 to improve the scalability and flexibility of the Hadoop ecosystem.
- YARN separates the resource management and job scheduling functions into separate daemons.
- The ResourceManager is responsible for managing resources in the cluster, while the ApplicationMaster is responsible for managing the execution of a single application.
- YARN allows multiple data processing engines to run on the same Hadoop cluster, enabling users to run different types of workloads simultaneously.
- YARN is designed to be scalable, efficient, and flexible, allowing users to dynamically allocate resources and run a wide range of applications on a Hadoop cluster.

Running MRv1 in YARN

- MRv1, also known as MapReduce version 1, is a framework for processing large data sets in parallel across a cluster of computers.
- YARN, or Yet Another Resource Negotiator, is a resource management layer in Hadoop that allows multiple data processing engines to share a common cluster.
- To run MRv1 in YARN, the following steps can be taken:
 1. Ensure that the Hadoop cluster is properly configured to use YARN as the resource manager.
 2. Set the `mapreduce.framework.name` property to `yarn` in the `mapred-site.xml` configuration file.
 3. Submit the MapReduce job to the cluster using the `hadoop jar` command, specifying the input and output paths, as well as any other necessary job configuration options.
 4. Monitor the progress of the job using the YARN web UI or the `yarn application` command.
- Running MRv1 in YARN allows for more efficient resource utilization and better scalability compared to running MRv1 in standalone mode.

NoSQL Databases

- NoSQL originally refers to “non-SQL” or “non-relational” databases.
- NoSQL databases provide a mechanism for storage and retrieval of data that is modeled in means other than the tabular relations used in relational databases.
- The data structures used by NoSQL databases (e.g. key-value pair, wide column, graph, or document) are different from those used by default in relational databases, making some operations faster in NoSQL.
- The particular suitability of a given NoSQL database depends on the problem it must solve.
- NoSQL databases are interchangeably referred to as “nonrelational,” “NoSQL DBs,” or “non-SQL” to highlight the fact that they can handle huge volumes of rapidly changing, unstructured data in different ways than a relational (SQL) database with rows and tables.
- NoSQL databases are designed to be used across large distributed systems. They are notably much more scalable and much faster at handling very large data loads than traditional relational databases.
- NoSQL databases do not use the standard tabular relationships the relational databases employ.
- NoSQL databases (aka “not only SQL”) are non-tabular databases and store data differently than relational tables.
- NoSQL databases come in a variety of types based on their data model. The main types are document, key-value, wide-column, and graph.
- NoSQL databases provide flexible schemas and scale easily with large amounts of data and high user loads.

- Some popular NoSQL databases include Apache CouchDB, Elasticsearch, and Couchbase.

Introduction to NoSQL databases

NoSQL databases are non-relational databases that store and retrieve data in ways that do not involve the use of a fixed schema like traditional relational databases. Some of the key characteristics of NoSQL databases include:

1. **Schema-less:** NoSQL databases do not require a fixed schema and can handle unstructured and semi-structured data.
2. **Scalability:** NoSQL databases are designed to scale horizontally, making it easy to add more capacity by adding more servers.
3. **Flexibility:** NoSQL databases allow for flexible data modeling, making it easy to change the data model without having to make changes to the underlying database.
4. **High performance:** NoSQL databases are optimized for specific data access patterns, providing high performance for certain types of queries.

There are several types of NoSQL databases, including document databases, key-value stores, column-family stores, and graph databases. Each type of NoSQL database is designed to handle specific data access patterns and use cases.

NoSQL databases are commonly used in big data and real-time web applications, where the ability to handle large volumes of unstructured data and scale horizontally is important. Some popular NoSQL databases include MongoDB, Cassandra, and Redis.

MongoDB

MongoDB is a cross-platform document-oriented database program. It is classified as a NoSQL database program, which means that it uses a non-tabular data model for storing data. Instead of using tables and rows as in traditional relational databases, MongoDB uses JSON-like documents with optional schemas.

Some key features of MongoDB include:

1. **Document-based data model:** MongoDB stores data in flexible, JSON-like documents, meaning fields can vary from document to document and data structure can be changed over time.
2. **Ad hoc queries:** MongoDB supports field, range, and regular expression queries, and can also search within documents and arrays.
3. **Indexing:** MongoDB supports indexing any field in a document, including fields within arrays, to improve query performance.

4. **Aggregation:** MongoDB provides an aggregation framework for data analysis and transformation, as well as a MapReduce function for batch processing of data.
5. **Replication and high availability:** MongoDB provides built-in replication and automatic failover for high availability.
6. **Horizontal scalability:** MongoDB can be scaled horizontally through sharding, automatically distributing data across multiple servers.
7. **Flexible schema:** MongoDB's dynamic schema allows for changes to the data model without requiring changes to the underlying database structure.

MongoDB is widely used for its flexibility, scalability, and performance, and is commonly used for web and mobile applications, real-time analytics, and content management systems. It is available under the Server Side Public License, a free and open-source license.

Introduction to MongoDB

MongoDB is a cross-platform document-oriented database program. It is classified as a NoSQL database program, which means it does not use the traditional tabular relational database structure. Instead, it uses JSON-like documents with optional schemas. Some of the key features of MongoDB include:

1. **Document-based:** Data is stored in flexible, JSON-like documents, meaning fields can vary from document to document and data structure can be changed over time.
2. **Scalable:** MongoDB is horizontally scalable, meaning it can handle large amounts of data by spreading it across multiple servers.
3. **Flexible:** MongoDB's dynamic schema allows for easy modification of data structure without downtime.
4. **Expressive query language:** MongoDB's query language allows for powerful and expressive queries to retrieve and manipulate data.

MongoDB is used by many large companies and organizations, including Adobe, eBay, and MetLife. It is a popular choice for web and mobile applications due to its flexibility and scalability.

Data Types in MongoDB

MongoDB supports several data types, including:

1. **String:** This is the most commonly used data type to store data. Strings in MongoDB must be UTF-8 valid.
2. **Integer:** This type is used to store a numerical value. Integer can be a 32-bit or 64-bit, depending on the server architecture.
3. **Boolean:** This type is used to store a boolean (true/ false) value.
4. **Double:** This type is used to store floating point values.

5. **Min/ Max keys:** This type is used to compare a value against the lowest and highest BSON elements, respectively.
6. **Arrays:** This type is used to store arrays or list or multiple values into one key.
7. **Timestamp:** ctimestamp. This can be handy for recording when a document has been modified or added.
8. **Object:** This type is used to store embedded documents.
9. **Null:** This type is used to store a Null value.
10. **Symbol:** This type is used identically to a string; however, it's generally reserved for languages that use a specific symbol type.
11. **Date:** This type is used to store the current date or time in UNIX time format. You can specify your own date time by creating an object of Date and passing the day, month, year into it.
12. **Object ID:** This is a 12-byte hexadecimal number which assures the uniqueness of every document. You can provide your own 12-byte id or, if not provided, MongoDB will generate one for you.
13. **Binary data:** This type is used to store binary data.
14. **Code:** This type is used to store JavaScript code into the document.
15. **Regular expression:** This type is used to store regular expression.

These are the most commonly used data types in MongoDB. Each data type has its own specific use case and it is important to choose the right data type for the data being stored.

Creating Documents in MongoDB

MongoDB is a document-based database that stores data in flexible, JSON-like documents. Here are the steps to create a document in MongoDB:

1. Connect to the MongoDB server and specify the database and collection you want to use.
2. Use the `insertOne()` or `insertMany()` method to insert a single document or multiple documents into the collection.
3. The `insertOne()` method takes a single document as an argument, while the `insertMany()` method takes an array of documents as an argument.
4. The inserted document(s) will be assigned a unique `_id` field by MongoDB if it is not specified in the document.
5. You can check the inserted document(s) by using the `find()` method on the collection.

Here is an example of inserting a single document into a collection named `users` in a database named `mydb`:

```
const MongoClient = require('mongodb').MongoClient;
const uri = "mongodb+srv://<username>:<password>@cluster0.mongodb.net/mydb";
const client = new MongoClient(uri, { useNewUrlParser: true });
client.connect(err => {
  const collection = client.db("mydb").collection("users");
```

```

// Insert a single document
collection.insertOne({ name: "John", age: 25 }, (err, result) => {
  if (err) throw err;
  console.log(result.insertedCount);
  client.close();
});
});

```

And here is an example of inserting multiple documents into the same collection:

```

const MongoClient = require('mongodb').MongoClient;
const uri = "mongodb+srv://<username>:<password>@cluster0.mongodb.net/mydb";
const client = new MongoClient(uri, { useNewUrlParser: true });
client.connect(err => {
  const collection = client.db("mydb").collection("users");
  // Insert multiple documents
  collection.insertMany([{ name: "Jane", age: 28 }, { name: "Bob", age: 32 }], (err, result) => {
    if (err) throw err;
    console.log(result.insertedCount);
    client.close();
  });
});

```

Updating Documents in MongoDB

MongoDB provides several methods to update documents in a collection. Here are some key points to remember when updating documents in MongoDB:

1. The `updateOne()` method updates a single document that matches the specified filter. If multiple documents match the filter, only the first document is updated.
2. The `updateMany()` method updates all documents that match the specified filter.
3. The `$set` operator is used to update specific fields in a document. If the field does not exist, it will be added to the document.
4. The `$inc` operator is used to increment the value of a field by a specified amount.
5. The `$push` operator is used to add an element to an array field.
6. The `$pull` operator is used to remove an element from an array field.
7. The `replaceOne()` method replaces a single document that matches the specified filter. The replacement document must contain all the fields that are required for the document to be valid.
8. The `update()` method is deprecated and should not be used. Instead, use the `updateOne()`, `updateMany()`, or `replaceOne()` methods.

These are some of the key points to remember when updating documents in MongoDB. It is important to carefully consider the update operation and use the appropriate method and operators to achieve the desired result.

Deleting Documents in MongoDB

MongoDB provides several methods to delete documents from a collection:

1. **deleteOne()**: This method deletes a single document that matches the specified filter. If multiple documents match the filter, only the first document is deleted.
2. **deleteMany()**: This method deletes all documents that match the specified filter.
3. **findOneAndDelete()**: This method finds a single document that matches the specified filter and deletes it, returning the original document.

Here is an example of how to use the **deleteOne()** method to delete a document from a collection:

```
db.collectionName.deleteOne({field: value})
```

In this example, **collectionName** is the name of the collection from which you want to delete the document, **field** is the name of the field you want to use to filter the documents, and **value** is the value of the field that the document must have to be deleted.

It is important to note that deleting documents from a collection is a permanent action and cannot be undone. Therefore, it is recommended to use caution when deleting documents from a collection.

Querying Documents in MongoDB

MongoDB is a NoSQL database that stores data in the form of documents. These documents are stored in collections, which are similar to tables in a relational database. To query documents in MongoDB, you can use the **find()** method. Here are some key points to remember when querying documents in MongoDB:

1. The **find()** method is used to query documents in a collection. It takes a query filter as an argument and returns a cursor to the documents that match the query.
2. You can specify the fields to return in the query results by passing a projection document as the second argument to the **find()** method.
3. You can use query operators such as **\$gt**, **\$lt**, **\$in**, and **\$or** to specify conditions in the query filter.
4. You can use the **sort()**, **skip()**, and **limit()** methods to control the order and number of documents returned by the query.
5. You can use the **count()** method to count the number of documents that match a query.
6. You can use the **explain()** method to obtain information about the query execution plan.

Indexing in MongoDB

- MongoDB uses indexing to make query processing more efficient. If there is no indexing, then MongoDB must scan every document in the collection and retrieve only those documents that match the query.
- Indexes are special data structures that store information related to the documents, making it easier for MongoDB to find the right data file. The indexes are ordered by the value of the field specified in the index.
- MongoDB provides a method called `createIndex()` that allows the user to create an index.
- MongoDB supports various types of indexes, including single field, compound, and multikey indexes.
- Multikey indexes are used to index the content stored in arrays. If you index a field that holds an array value, MongoDB creates separate index entries for every element of the array.

Aggregation in MongoDB

- Aggregation in MongoDB is the process of grouping and analyzing data to obtain a desired result.
- The aggregation framework in MongoDB provides a way to perform complex data manipulation and analysis operations on collections of documents.
- The aggregation pipeline is a series of stages, where each stage transforms the documents as they pass through.
- The most common aggregation operations include filtering, grouping, projecting, and sorting documents.
- MongoDB provides several aggregation operators, such as `$match`, `$group`, `$project`, and `$sort`, to perform these operations.
- Aggregation can be performed using the `aggregate()` method on a collection or using the `db.collection.aggregate()` command.
- The aggregation framework is optimized for performance and can handle large amounts of data efficiently.
- Aggregation can also be performed using MapReduce, but the aggregation framework is generally faster and easier to use.

Capped Collections in MongoDB

- Capped collections are fixed-size collections that support high-throughput operations that insert and retrieve documents based on insertion order.
- Capped collections work in a way similar to circular buffers: once a collection fills its allocated space, it makes room for new documents by overwriting the oldest documents in the collection.
- To create a capped collection, you must specify the size of the collection in bytes. Optionally, you can also specify the maximum number of documents for the collection.

- Capped collections automatically maintain insertion order for the documents in the collection. This means that you can use natural order queries to retrieve documents in the order they were inserted.
- Capped collections have some limitations. For example, you cannot delete documents from a capped collection, and you cannot update documents in a way that would increase their size.
- Capped collections are ideal for applications that require fast, high-throughput insertion and retrieval of documents in insertion order, such as log data or real-time data feeds.

Spark

Apache Spark is an open-source distributed general-purpose cluster-computing framework. It provides an interface for programming entire clusters with implicit data parallelism and fault tolerance. Some of the key features of Spark include:

1. **Speed:** Spark is designed to be fast, both for batch processing and for iterative algorithms. It can run programs up to 100x faster than Hadoop MapReduce in memory, or 10x faster on disk.
2. **Ease of Use:** Spark has easy-to-use APIs for operating on large datasets. It supports multiple languages, including Python, R, Scala, and SQL.
3. **Generality:** Spark combines SQL, streaming, and complex analytics in a single engine. This makes it easy to build and combine different types of data processing and analytics workloads.
4. **Runs Everywhere:** Spark runs on Hadoop, Mesos, standalone, or in the cloud. It can access diverse data sources, including HDFS, Cassandra, HBase, and S3.
5. **Advanced Analytics:** Spark includes built-in libraries for machine learning, graph processing, and stream processing. These libraries make it easy to build sophisticated analytics applications.

Spark is widely used in big data processing, data science, and machine learning. It has a large and active community, with many contributors and users. It is a powerful tool for processing large datasets and building advanced analytics applications.

Installing Spark

1. Download the latest version of Spark from the Apache Spark website.
2. Unpack the downloaded file to a directory of your choice.
3. Set the environment variable `SPARK_HOME` to the directory where you unpacked Spark.
4. Add the `$SPARK_HOME/bin` directory to your `PATH` environment variable.
5. Test the installation by running the command `spark-shell` in a terminal. If the installation was successful, you should see the Spark shell prompt.

6. Optionally, you can also configure Spark by editing the `spark-defaults.conf` file located in the `$SPARK_HOME/conf` directory. This file contains default configuration options for Spark.

Spark Applications

- Spark Applications consist of a driver process and a set of executor processes.
- The driver process runs your `main ()` function, sits on a node in the cluster, and is responsible for three things: maintaining information about the Spark Application; responding to a user's program or input; and analyzing, distributing, and scheduling work across the executors.
- Spark applications run as independent sets of processes on a cluster, coordinated by the driver program.
- The driver consists of your program, like a C# console app, and a Spark session. The Spark session takes your program and divides it into smaller tasks that are handled by the executors.
- A Spark application runs as independent processes, coordinated by the `SparkSession` object in the driver program. The resource or cluster manager assigns tasks to workers, one task per partition. A task applies its unit of work to the dataset in its partition and outputs a new partition dataset.
- For an in-depth overview of the API, start with the RDD programming guide and the SQL programming guide, or see "Programming Guides" menu for other components. For running applications on a cluster, head to the deployment overview.
- Spark applications run as independent sets of processes on a pool, coordinated by the `SparkContext` object in your main program, called the driver program. The `SparkContext` can connect to the cluster manager, which allocates resources across applications. The cluster manager is Apache Hadoop YARN.

Jobs in Spark

- A job in Apache Spark is a parallel computation consisting of multiple tasks that gets spawned in response to a Spark action.
- Jobs are divided into stages, which are a collection of tasks that can run in parallel.
- Each stage contains tasks that perform the same computation, but on different data partitions.
- Jobs are triggered by actions, such as `count()`, `collect()`, or `save()`, which return a value or produce a side effect.
- Jobs are submitted to the Spark cluster manager, which is responsible for scheduling and distributing the tasks across the cluster.
- The progress of a job can be monitored through the Spark web UI, which displays information about completed and active stages, as well as the

status of individual tasks.

- Jobs can be cancelled by the user or terminated by the cluster manager if they exceed a specified time limit or consume too much resources.
- The performance of a job can be optimized by tuning various parameters, such as the level of parallelism, the amount of memory allocated to each executor, and the choice of data serialization format.

Stages and Tasks in Spark

Apache Spark is a distributed computing system that processes large data sets in parallel. The processing of data in Spark is divided into stages, and each stage is further divided into tasks.

1. **Stages:** A stage in Spark is a collection of tasks that can be executed in parallel. Stages are created based on the dependencies between the Resilient Distributed Datasets (RDDs) in the application. Each stage contains tasks that perform the same computation on different partitions of the data.
2. **Tasks:** A task in Spark is a unit of work that is sent to an executor to be processed. Each task processes a single partition of the data. The number of tasks in a stage is equal to the number of partitions in the RDD being processed.
3. **Shuffling:** When the data needs to be redistributed among the partitions, a shuffle operation is performed. This can happen, for example, when data is grouped by key. Shuffling can be an expensive operation, and Spark tries to minimize the amount of data that needs to be shuffled.
4. **DAG Scheduler:** The Directed Acyclic Graph (DAG) Scheduler is responsible for dividing the computation into stages and tasks. It creates a DAG of stages based on the dependencies between the RDDs and determines the order in which the stages should be executed.
5. **Job:** A job in Spark is a sequence of stages that are required to compute the result of an action. When an action is called on an RDD, the DAG Scheduler creates a job to compute the result. The job is then divided into stages, and the stages are further divided into tasks.

In summary, the processing of data in Spark is divided into stages, and each stage is further divided into tasks. The DAG Scheduler is responsible for dividing the computation into stages and tasks, and a job is a sequence of stages that are required to compute the result of an action. Shuffling can occur when data needs to be redistributed among the partitions.

Resilient Distributed Databases in Spark

Resilient Distributed Datasets (RDD) is a fundamental data structure of Spark. It is an immutable distributed collection of objects. Each dataset in RDD is

divided into logical partitions, which may be computed on different nodes of the cluster. RDDs can contain any type of Python, Java, or Scala objects, including user-defined classes.

RDDs are reliable and memory-efficient when it comes to parallel processing. By storing and processing data in RDDs, Spark speeds up MapReduce processes.

At the core, an RDD is an immutable distributed collection of elements of your data, partitioned across nodes in your cluster that can be operated in parallel with a low-level API that offers transformations and actions.

Spark can create distributed datasets from any storage source supported by Hadoop, including your local file system, HDFS, Cassandra, HBase, Amazon S3, etc. Spark supports text files, SequenceFiles, and any other Hadoop Input-Format. Text file RDDs can be created using SparkContext's `textFile` method.

Anatomy of a Spark Job Run

1. **Client Mode:** In client mode, the driver program runs on the client machine, and the application master is only used for requesting resources from the cluster manager.
2. **Cluster Mode:** In cluster mode, the driver program runs on a worker node in the cluster, and the client can go away after submitting the application.
3. **Job Submission:** When an action is called on an RDD, a job is submitted to the Spark scheduler.
4. **Stages:** A job is divided into stages, where each stage contains a sequence of narrow transformations that can be pipelined together.
5. **Tasks:** Each stage is further divided into tasks, where each task corresponds to a partition of the input data.
6. **Task Scheduling:** The scheduler assigns tasks to available executors based on data locality and available resources.
7. **Task Execution:** Each task is executed on an executor, reading its input data, applying the transformations, and writing its output data.
8. **Shuffling:** Wide transformations require data to be shuffled between executors, which can be a costly operation.
9. **Result Collection:** Once all tasks have completed, the result is collected and returned to the driver program.

Spark on YARN

- Apache Spark is a fast and general-purpose cluster computing system.
- It provides high-level APIs in Java, Scala, Python, and R, and an optimized engine that supports general computation graphs for data analysis.
- YARN (Yet Another Resource Negotiator) is the resource management layer of Hadoop.
- Spark can run on YARN, allowing it to take advantage of the resource management capabilities of YARN.

- When running Spark on YARN, each Spark executor runs as a YARN container.
- YARN allocates resources (CPU, memory, etc.) to the Spark application based on the configured resource allocation policies.
- This allows multiple Spark applications to run concurrently on the same cluster, sharing resources fairly.
- To run Spark on YARN, the `spark-submit` script must be configured to use the `yarn` master.
- The `spark-submit` script takes care of uploading the Spark application JAR and any dependencies to the Hadoop Distributed File System (HDFS), and launching the application on the YARN cluster.
- Running Spark on YARN provides several benefits, including dynamic resource allocation, data locality, and integration with other Hadoop ecosystem tools.

SCALA

Scala is a general-purpose, high-level, multi-paradigm programming language. It is a strong statically typed language that supports both object-oriented programming and functional programming. Scala is designed to be concise, and many of its design decisions are aimed to address criticisms of Java.

Scala programs can be converted to bytecodes and can run on the JVM (Java Virtual Machine). Scala's static types help avoid bugs in complex applications, and its JVM and JavaScript runtimes let you build high-performance systems with easy access to huge ecosystems of libraries.

- Scala is a general-purpose, high-level, multi-paradigm programming language.
- It is a strong statically typed language.
- Supports both object-oriented programming and functional programming.
- Designed to be concise.
- Many of its design decisions are aimed to address criticisms of Java.
- Scala programs can be converted to bytecodes and can run on the JVM (Java Virtual Machine).
- Scala's static types help avoid bugs in complex applications.
- Its JVM and JavaScript runtimes let you build high-performance systems with easy access to huge ecosystems of libraries.

Introduction to Scala

Scala is a modern, multi-paradigm programming language designed to express common programming patterns in a concise, elegant, and type-safe way. It smoothly integrates features of object-oriented and functional languages.

Some key features of Scala include: - Scala is a statically typed language, which means that the type of a variable is checked at compile-time. - Scala is both object-oriented and functional. Every value is an object and every operation is a

method call. - Scala has a concise and expressive syntax, allowing developers to write code in a more readable and maintainable way. - Scala has a sophisticated type system, supporting features such as generic classes, variance annotations, and higher-order types. - Scala has built-in support for concurrency and parallelism, making it easier to write programs that can take advantage of multiple cores and processors. - Scala is interoperable with Java, allowing developers to use existing Java libraries and frameworks.

Scala is used by many companies, including Twitter, LinkedIn, and Netflix, for a wide range of applications, from web development to data analysis. It is also a popular language for teaching and research in computer science.

Classes and Objects in Scala

- **Classes** in Scala are blueprints for creating objects. They can contain methods, values, variables, types, objects, traits, and classes which are collectively called members.
- **Objects** in Scala are instances of classes. They are created using the **new** keyword followed by the constructor of the class.
- A **constructor** is a special method that is used to initialize the object. The primary constructor is defined within the class signature, while additional constructors can be defined using the **def this(...)** syntax.
- **Members** of a class can be accessed using the dot **.** notation. For example, if **obj** is an instance of a class with a member **x**, then **obj.x** refers to the value of **x** for that instance.
- **Methods** in Scala are defined using the **def** keyword. They can take parameters and can return a value. Methods can be called on an instance of a class using the dot **.** notation.
- **Inheritance** in Scala allows a class to inherit members from a superclass. This is done using the **extends** keyword. A subclass can override members of the superclass using the **override** keyword.
- **Traits** in Scala are similar to interfaces in other languages. They define a set of abstract methods that must be implemented by any class that mixes in the trait. Traits can also contain concrete methods and fields.
- **Companion objects** in Scala are objects that have the same name as a class and are defined in the same source file. They can access private members of the class and are often used to define factory methods for the class.
- **Case classes** in Scala are special classes that are used to model immutable data. They automatically generate several useful methods such as **equals**, **hashCode**, and **toString**. Case classes can be created without using the **new** keyword.

- **Singleton objects** in Scala are objects that are defined using the `object` keyword. They are used to define global values and methods and can be accessed directly without creating an instance. They are similar to static members in other languages.

Basic Types and Operators in Scala

Scala has a rich set of built-in data types and operators. Here are some of the basic types and operators in Scala:

1. **Numeric Types:** Scala has several numeric types, including `Byte`, `Short`, `Int`, `Long`, `Float`, and `Double`. These types represent 8-bit, 16-bit, 32-bit, and 64-bit signed integers, as well as 32-bit and 64-bit floating-point numbers, respectively.
2. **Boolean Type:** The `Boolean` type in Scala has two values: `true` and `false`. Boolean expressions are used to make decisions in the code.
3. **Character Type:** The `Char` type in Scala represents a single 16-bit Unicode character.
4. **String Type:** The `String` type in Scala represents a sequence of characters. Strings are immutable in Scala, meaning that once a string is created, it cannot be changed.
5. **Arithmetic Operators:** Scala has several arithmetic operators, including `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), and `%` (modulus).
6. **Relational Operators:** Scala has several relational operators, including `==` (equal to), `!=` (not equal to), `>` (greater than), `<` (less than), `>=` (greater than or equal to), and `<=` (less than or equal to).
7. **Logical Operators:** Scala has several logical operators, including `&&` (logical AND), `||` (logical OR), and `!` (logical NOT).
8. **Bitwise Operators:** Scala has several bitwise operators, including `&` (bitwise AND), `|` (bitwise OR), `^` (bitwise XOR), `~` (bitwise NOT), `<<` (left shift), `>>` (right shift), and `>>>` (unsigned right shift).

These are some of the basic types and operators in Scala. They provide the foundation for building more complex programs and algorithms in the language.

Built-in Control Structures in Scala

Scala has several built-in control structures that allow you to control the flow of your program. These include:

1. **If-else statements:** These allow you to execute different code blocks depending on whether a condition is true or false.

```

if (x > 0) {
  println("x is positive")
} else {
  println("x is not positive")
}

```

2. **While loops:** These allow you to repeatedly execute a code block while a condition is true.

```

while (x > 0) {
  println(x)
  x -= 1
}

```

3. **For loops:** These allow you to iterate over a collection of elements.

```

for (x <- 1 to 10) {
  println(x)
}

```

4. **Match expressions:** These allow you to pattern match on values and execute different code blocks depending on the value.

```

x match {
  case 1 => println("x is 1")
  case 2 => println("x is 2")
  case _ => println("x is something else")
}

```

These are some of the built-in control structures in Scala that you can use to control the flow of your program. They are similar to control structures in other programming languages, but with some syntactic differences.

Functions and Closures in Scala

- In Scala, functions are first-class values, meaning they can be assigned to variables, passed as arguments to other functions, and returned as values from other functions.
- A function literal is an expression that defines an anonymous function. It is written using the `=>` symbol, with the parameters on the left and the function body on the right.
- A closure is a function that references variables from its enclosing scope. The function and the referenced variables together form a closure.
- Closures allow you to create functions that have behavior that depends on data that is not passed as a parameter.
- In Scala, closures are automatically created when a function literal references a variable from its enclosing scope.
- Closures are useful for creating functions that need to maintain state between invocations, such as in functional programming patterns like curry-

ing and partial application.

- Scala also provides support for higher-order functions, which are functions that take other functions as arguments or return them as results. Higher-order functions are commonly used in functional programming to create more modular and reusable code.

Inheritance in Scala

Inheritance is a fundamental concept in object-oriented programming that allows the creation of hierarchical classifications. Scala, being an object-oriented language, supports inheritance. Here are some key points to remember about inheritance in Scala:

1. Inheritance allows a class to reuse code from another class. In Scala, a class can inherit from another class using the **extends** keyword.
2. A subclass inherits all the non-private members (fields and methods) of its superclass.
3. Constructors are not inherited by subclasses, but a subclass constructor can call a superclass constructor.
4. Scala supports single class inheritance, meaning a class can only inherit from one superclass.
5. Scala also supports multiple inheritance of behavior through traits. A class can inherit behavior from multiple traits using the **with** keyword.
6. A subclass can override a non-private method of its superclass by using the **override** keyword.
7. A subclass can also override a field of its superclass, but this is not common practice.
8. Inheritance creates an **is-a** relationship between the subclass and the superclass. For example, if class **Dog** extends class **Animal**, then a **Dog** is an **Animal**.

These are some of the key points to remember about inheritance in Scala. It is a powerful tool that allows for code reuse and the creation of hierarchical classifications. It is important to use inheritance appropriately and understand its implications in order to write effective and maintainable code.

Hadoop Eco System Frameworks, Pig, Hive and HBase

- **Hadoop** includes several additional modules that provide additional functionality, such as **Hive**, **Pig**, and **HBase**.
- **HBase** is a scalable, distributed database that supports structured data storage for large tables.
- **Hive** is a data warehouse infrastructure that provides data summarization and ad-hoc querying. Its query language is called **HQL (Hive Query Language)**.
- **Pig** is a high-level data-flow language and execution framework for parallel computation. It helps to achieve ease of programming and optimization.

- **HBase** is a distributed column-oriented database built on top of the **HDFS (Hadoop Distributed File System)**. It is an open-source project and horizontally scalable. HBase is a data model that is similar to Google's big table designed to provide quick random access to huge amounts of structured data.

Hadoop Eco System Frameworks

Hadoop is a framework that enables processing of large data sets which reside in the form of clusters. Being a framework, Hadoop is made up of several modules that are supported by a large ecosystem of technologies .

- **Introduction:** Hadoop Ecosystem is a platform or a suite which provides various services to solve the big data problems. It includes Apache projects and various commercial tools and solutions .
- **Major Elements:** There are four major elements of Hadoop i.e. HDFS, MapReduce, YARN, and Hadoop Common .
- **Hadoop Distributed File System (HDFS):** The core component of the Hadoop ecosystem is a Hadoop distributed file system (HDFS). HDFS is a distributed file system that has the capability to store a large stack of data sets .
- **Apache Hadoop:** The Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models. It is designed to scale up from single servers to thousands of machines, each offering local computation and storage .
- **Open-Source:** Hadoop is an open-source software framework for storing data and running applications on clusters of commodity hardware. It provides massive storage for any kind of data, enormous processing power and the ability to handle virtually limitless concurrent tasks or jobs .

Applications on Big Data using Pig

Pig is a high-level platform for creating MapReduce programs used with Hadoop. It is designed to handle any kind of data, making it an ideal tool for analyzing large datasets. Here are some of the applications of Big Data using Pig:

1. **Data processing:** Pig can be used to perform data processing tasks such as filtering, grouping, and sorting on large datasets.
2. **Data analysis:** Pig can be used to perform data analysis tasks such as calculating averages, sums, and counts on large datasets.
3. **Data cleansing:** Pig can be used to clean and prepare data for analysis by removing null values, duplicates, and outliers.

4. **Data integration:** Pig can be used to integrate data from multiple sources by joining and merging datasets.
5. **Data storage:** Pig can be used to store data in a structured format, making it easier to query and analyze.
6. **Data visualization:** Pig can be used to generate visualizations of data, making it easier to understand and interpret.
7. **Machine learning:** Pig can be used to prepare data for machine learning algorithms by performing tasks such as feature extraction and data normalization.

Overall, Pig is a powerful tool for working with Big Data, and its applications are numerous and varied. It is an essential tool for anyone working with large datasets.

Applications on Big Data using Hive

Hive is a data warehousing and SQL-like query language for Apache Hadoop, which enables data summarization, querying, and analysis of large datasets. Some of the applications of Big Data using Hive are:

1. **Data Processing:** Hive can be used to process structured and semi-structured data in Hadoop. It provides an SQL-like interface to query data stored in various databases and file systems that integrate with Hadoop.
2. **Data Analysis:** Hive can be used for data analysis, such as finding trends and patterns in large datasets. It supports complex data analysis using custom MapReduce scripts and user-defined functions.
3. **Data Mining:** Hive can be used for data mining, such as finding correlations and associations between different data elements. It supports data mining algorithms such as k-means clustering and association rule mining.
4. **Data Visualization:** Hive can be used for data visualization, such as creating charts and graphs to represent data. It can be integrated with data visualization tools such as Tableau and QlikView to create interactive dashboards and reports.
5. **Data Reporting:** Hive can be used for data reporting, such as generating reports and summaries of large datasets. It supports various reporting formats such as CSV, Excel, and PDF.

Overall, Hive is a powerful tool for Big Data applications, providing a flexible and scalable solution for data processing, analysis, mining, visualization, and reporting.

Applications on Big Data using HBase

HBase is a distributed, scalable, and big data store that is used to store large amounts of data in a tabular format. It is an open-source, non-relational, and column-oriented database that is built on top of the Hadoop Distributed File System (HDFS). Some of the applications of Big Data using HBase are:

1. **Real-time data analytics:** HBase can be used to perform real-time data analytics on large datasets. It can handle large amounts of data and provide fast and random read/write access to the data.
2. **Data warehousing:** HBase can be used as a data warehouse to store large amounts of data. It can handle structured, semi-structured, and unstructured data and provide fast access to the data.
3. **Log storage and analysis:** HBase can be used to store and analyze log data. It can handle large amounts of log data and provide fast access to the data.
4. **Recommendation systems:** HBase can be used to build recommendation systems. It can handle large amounts of data and provide fast access to the data.
5. **Fraud detection:** HBase can be used to detect fraud in real-time. It can handle large amounts of data and provide fast access to the data.
6. **Social media analysis:** HBase can be used to analyze social media data. It can handle large amounts of data and provide fast access to the data.
7. **IoT data storage and analysis:** HBase can be used to store and analyze IoT data. It can handle large amounts of data and provide fast access to the data.

Pig

- The pig (*Sus domesticus*), often called swine, hog, or domestic pig when distinguishing from other members of the genus *Sus*, is an omnivorous, domesticated, even-toed, hoofed mammal. It is variously considered a subspecies of *Sus scrofa* (the wild boar or Eurasian boar) or a distinct species.
- Pigs are stout-bodied, short-legged, omnivorous mammals, with thick skin usually sparsely coated with short bristles. Their hooves have two functional and two nonfunctional digits.
- Domestic North American pigs originated from wild stocks still found in European, Asian, and North African forests.
- Any of various mammals of the family Suidae, having short legs, hooves with two weight-bearing toes, bristly hair, and a cartilaginous snout used for digging, including the domesticated hog (*Sus scrofa* subsp. *domestica* syn. *S. domesticus*) and wild species such as the bushpig.

Pig - Introduction to PIG

1. Pig is a high-level platform for creating MapReduce programs used with Hadoop.
2. It is a data flow language that abstracts the programming from the Java MapReduce idiom into a notation which makes MapReduce programming high level, similar to that of SQL for RDBMS systems.
3. Pig Latin is the language used to express data flows in Pig.
4. Pig Latin scripts are automatically optimized by the Pig runtime, so the programmer does not have to worry about the execution plan.
5. Pig can be used to process structured, semi-structured, and unstructured data.
6. Pig can be used interactively or in batch mode.
7. Pig can be extended using User Defined Functions (UDFs) written in Java, Python, or other languages.
8. Pig is an Apache open-source project.

Execution Modes of Pig

Pig is a high-level platform for creating MapReduce programs used with Hadoop. It has two execution modes:

1. **Local Mode:** In this mode, Pig runs on a single machine without requiring Hadoop or HDFS. It is used for development and testing of Pig scripts.
2. **MapReduce Mode:** In this mode, Pig runs on a Hadoop cluster and requires HDFS. It is used for processing large data sets in a distributed environment.

Both modes can be invoked by specifying the appropriate command line option when running Pig. For example, to run Pig in local mode, the command would be `pig -x local`. To run Pig in MapReduce mode, the command would be `pig` or `pig -x mapreduce`.

Comparison of Pig with Databases

Pig is a high-level platform for creating MapReduce programs used with Hadoop. It is similar to databases in some ways, but there are also some key differences between Pig and databases. Here are some points of comparison:

1. **Data storage:** Databases store data in tables, while Pig processes data stored in the Hadoop Distributed File System (HDFS).
2. **Data processing:** Databases use SQL for data processing, while Pig uses its own language, Pig Latin, for data processing.
3. **Data types:** Databases have a fixed schema and support a limited number of data types, while Pig has a flexible schema and can handle complex data types such as maps and tuples.
4. **Data analysis:** Databases are designed for online transaction processing (OLTP) and are optimized for fast data retrieval, while Pig is designed for

batch processing and is optimized for data analysis.

5. **Scalability:** Databases can be scaled vertically by adding more resources to a single machine, while Pig can be scaled horizontally by adding more machines to a cluster.

In summary, Pig and databases have some similarities, but they also have some key differences in terms of data storage, processing, types, analysis, and scalability. Depending on the specific use case, one may be more suitable than the other.

Grunt in Pig

- Grunt is a shell command in Apache Pig.
- It is mainly used to write Pig Latin scripts.
- Pig scripts can be executed with Grunt shell, which is a native shell provided by Apache Pig to execute Pig queries .
- Grunt shell can also be used to invoke any shell commands using `sh` and `fs` .
- However, using `sh` command from the Grunt shell, we cannot execute the commands that are a part of the shell environment (ex – `cd`) .

Pig Latin

Pig Latin is a language game in which words in English are altered. The objective is to conceal the words from others not familiar with the rules. The reference to Latin is a deliberate misnomer, as it is simply a form of jargon, used only for its English connotations as a strange and foreign-sounding language.

Here are the rules for translating English words into Pig Latin: 1. If a word begins with a consonant, all letters before the initial vowel are placed at the end of the word sequence. Then, “ay” is added. For example, “pig” would become “igpay”. 2. If a word begins with a vowel, just add “way” at the end. For example, “eat” would become “eatway”. 3. If a word has no vowels, just add “ay” at the end. For example, “my” would become “myay”.

Pig Latin is often used by children as a way to communicate in a way that adults cannot understand. It is also used by adults for humorous effect or to encode messages. It is not a true language, but rather a simple code that can be easily learned and used.

User Defined Functions in Pig

- User Defined Functions (UDFs) in Pig allow users to write their own functions to perform operations on data that are not supported by built-in Pig functions.
- UDFs can be written in Java, Python, Ruby, and other languages.
- UDFs can be used in Pig scripts by registering the JAR file containing the UDF and using the `DEFINE` keyword to create an alias for the function.

- UDFs can be used in expressions, filters, and other operations in a Pig script.
- UDFs can be shared and reused by other users.
- UDFs provide flexibility and extensibility to Pig, allowing users to perform custom operations on data.

Data Processing Operators in Pig

Pig is a high-level platform for creating MapReduce programs used with Hadoop. It includes a language called Pig Latin for expressing data analysis programs. Pig Latin includes several data processing operators that can be used to perform various data manipulation tasks. Here are some of the most commonly used data processing operators in Pig:

1. **LOAD**: This operator is used to load data from the file system into a Pig relation. The data can be in various formats such as text, binary, or sequence files.
2. **STORE**: This operator is used to store the data of a Pig relation into the file system. The data can be stored in various formats such as text, binary, or sequence files.
3. **FILTER**: This operator is used to filter out tuples from a relation based on a specified condition.
4. **FOREACH**: This operator is used to generate a new relation by applying a transformation to each tuple of an input relation.
5. **GROUP**: This operator is used to group the tuples of a relation based on one or more fields.
6. **JOIN**: This operator is used to join two or more relations based on a common field.
7. **ORDER**: This operator is used to sort the tuples of a relation based on one or more fields.
8. **DISTINCT**: This operator is used to remove duplicate tuples from a relation.
9. **LIMIT**: This operator is used to limit the number of tuples in a relation.
10. **UNION**: This operator is used to combine the tuples of two or more relations into a single relation.

These are some of the most commonly used data processing operators in Pig. They can be used in various combinations to perform complex data manipulation tasks. It is important to note that Pig Latin is a procedural language, and the order in which the operators are applied can affect the final result. Therefore, it is important to carefully plan the sequence of operations when writing Pig scripts.

Hive

- Hive is a data warehousing and SQL-like query language for data stored in Hadoop files.
- Hive enables data summarization, querying, and analysis of data.
- Hive queries are written in HiveQL, which is a query language similar to SQL.
- Hive allows you to project structure on largely unstructured data.
- Hive Metastore (HMS) provides a central repository of metadata that can easily be analyzed to make informed, data-driven decisions, and therefore it is a critical component of many data lake architectures.
- Hive is built on top of Apache Hadoop and supports storage on S3, ADLS, GS, etc. through HDFS.

Apache Hive architecture

Apache Hive is a distributed, fault-tolerant data warehouse system that enables analytics at a massive scale. It is an open-source data warehousing tool for performing distributed processing and data analysis. It was developed by Facebook to reduce the work of writing the Java MapReduce program. Apache Hive uses a Hive Query language, which is a declarative language similar to SQL.

The major components of Apache Hive are: - **Hive clients** - **Hive services** - **Processing framework and Resource Management** - **Distributed Storage**

The key components of the Apache Hive architecture are: - **Hive Server 2**: The Hive Server 2 accepts incoming requests from users and applications and creates an execution plan and auto generates a YARN job to process SQL queries. - **Hive Query Language (HQL)** - **External Apache Hive Metastore**: Hive Metastore (HMS) provides a central repository of metadata that can easily be analyzed to make informed, data driven decisions, and therefore it is a critical component of many data lake architectures. - **Hive Beeline Shell**

Installing Hive

1. Hive is a data warehousing and SQL-like query language for Hadoop.
2. To install Hive, you must first have Hadoop installed and configured on your system.
3. Download the latest stable release of Hive from the Apache Hive website.
4. Unpack the downloaded tarball and move the extracted directory to a location of your choice.
5. Set the environment variable `HIVE_HOME` to the location of the Hive installation.
6. Add the Hive `bin` directory to your `PATH` environment variable.
7. Configure Hive by editing the `hive-site.xml` file located in the `conf` directory of the Hive installation.
8. Start the Hive shell by running the `hive` command.

9. Verify that Hive is installed and working correctly by running a simple query, such as `SHOW TABLES;`.

Hive Shell

- Hive shell is a command line interface (CLI) for Apache Hive.
- It is used to interact with Hive and execute HiveQL commands.
- HiveQL is a SQL-like language used to query and manage data stored in Hive.
- To start the Hive shell, type `hive` in the command line and press enter.
- Once in the Hive shell, you can execute HiveQL commands by typing them and pressing enter.
- You can exit the Hive shell by typing `exit` or `quit` and pressing enter.
- The Hive shell also supports command history, which allows you to view and execute previously entered commands.
- The Hive shell is useful for ad-hoc querying and data exploration, but for more complex tasks, it is recommended to use other Hive interfaces such as the HiveServer2 or the Beeline CLI.

Hive Services

Hive is a data warehousing and SQL-like query language for Hadoop. It provides a mechanism to project structure onto data in Hadoop and to query that data using a SQL-like language called HiveQL. Hive services include:

1. **HiveServer2:** A service that provides a Thrift interface and a JDBC/ODBC server for clients to connect to and execute queries.
2. **Hive Metastore:** A service that stores metadata for Hive tables and partitions in a relational database and provides clients with a Thrift API to interact with this metadata.
3. **Hive Web Interface:** A web-based user interface for Hive that allows users to submit queries and view query results.
4. **Hive CLI:** A command line interface for Hive that allows users to interact with Hive from the command line.
5. **Beeline:** A JDBC client for HiveServer2 that can be used to execute queries from the command line.

These services work together to provide a comprehensive solution for data warehousing and querying in Hadoop. They allow users to interact with data stored in Hadoop using familiar SQL-like syntax, making it easier to integrate Hadoop into existing data analysis workflows.

Hive Metastore

- Hive Metastore (HMS) is a service that stores metadata related to Apache Hive and other services, in a backend RDBMS, such as MySQL or PostgreSQL .
- Impala, Spark, Hive, and other services share the metastore .

- The connections to and from HMS include HiveServer, Ranger, and the NameNode that represents HDFS .
- HMS is a central repository of metadata for Hive tables and partitions in a relational database, and provides clients (including Hive, Impala and Spark) access to this information using the metastore service API.
- HMS provides a central repository of metadata that can easily be analyzed to make informed, data-driven decisions, and therefore it is a critical component of many data lake architectures.
- Hive is built on top of Apache Hadoop and supports storage on S3, adls, gs etc though hdfs.
- Hive Metastore was developed as a part of Apache Hive, “a distributed, fault-tolerant data warehouse system that enables analytics at a massive scale”.
- Hive achieves this goal by being the storage point for all the meta-information about your data storages.

Comparison of Hive with traditional databases

Hive and traditional databases have several differences, some of which are:

- **Schema:** Hive applies schema on read time, meaning it does not verify the data when it is loaded, but rather when it is read. Traditional databases, on the other hand, apply schema on write time, meaning the table schema is enforced when data is loaded .
- **Scalability:** Hive is easily scalable at a low cost, while traditional databases are not as scalable and can be costly to scale up .
- **Data Manipulation:** Hive is based on Hadoop notation, which means it is write once and read many times. In traditional databases, data can be read and written multiple times. Record-level updates, insertions, and deletions are not possible in Hive, while they are possible in traditional databases .
- **Interface:** Hive supports an SQL-like interface, but it is not a full database. It can be better called a data warehouse instead of a database .

HiveQL

Hive Query Language (HiveQL) is a query language in Apache Hive for processing and analyzing structured data. It separates users from the complexity of Map Reduce programming. It reuses common concepts from relational databases, such as tables, rows, columns, and schema, to ease learning .

HiveQL provides the basic SQL like operations. SELECT statement is used to retrieve the data from a table . Hive provides Built-in operators for Data operations to be implemented on the tables present inside Hive warehouse .

Hive is built on top of Apache Hadoop and supports storage on S3, adls, gs etc though hdfs . Hive provides the necessary SQL abstraction to integrate SQL-like queries (HiveQL) into the underlying Java without the need to implement

queries in the low-level Java API . Since most data warehousing applications work with SQL-based querying languages, Hive aids portability of SQL-based applications to Hadoop .

Apache Hive is a data warehouse system for Apache Hadoop. Hive enables data summarization, querying, and analysis of data. Hive queries are written in HiveQL, which is a query language similar to SQL. Hive allows you to project structure on largely unstructured data .

Tables in Hive

Hive is a data warehousing and SQL-like query language for Hadoop. It allows users to create and manage tables in a relational database-like manner. Here are some key points about tables in Hive:

1. **Types of Tables:** Hive supports two types of tables: managed tables and external tables. Managed tables are created and managed by Hive, while external tables are created and managed by the user.
2. **Creating Tables:** Tables can be created in Hive using the `CREATE TABLE` statement. The syntax is similar to the `CREATE TABLE` statement in SQL.
3. **Loading Data:** Data can be loaded into Hive tables using the `LOAD DATA` statement. Data can be loaded from local files or from HDFS.
4. **Partitioning:** Hive supports partitioning of tables, which allows for faster querying of data. Partitioning is done by specifying one or more columns as partition columns when creating the table.
5. **Bucketing:** Hive also supports bucketing of tables, which is another way to improve query performance. Bucketing is done by specifying a column as the bucketing column and the number of buckets when creating the table.
6. **Altering Tables:** Tables in Hive can be altered using the `ALTER TABLE` statement. This allows for changes to the table structure, such as adding or dropping columns.
7. **Dropping Tables:** Tables can be dropped in Hive using the `DROP TABLE` statement. This will remove the table and all its data from the Hive metastore.

These are some of the key points about tables in Hive. It is important to understand these concepts when working with Hive tables.

Querying Data in Hive

Hive is a data warehousing and SQL-like query language for Hadoop. It allows users to query and analyze large datasets stored in Hadoop files. Here are some points to consider when querying data in Hive:

1. **HiveQL:** HiveQL is the query language used in Hive. It is similar to SQL and supports many of the same functions and syntax.

2. Tables: Data in Hive is organized into tables. Tables can be created and managed using HiveQL commands.
3. Partitions: Hive tables can be partitioned to improve query performance. Partitioning splits the data into smaller, more manageable chunks based on the values of one or more columns.
4. Joins: Hive supports several types of joins, including inner, left outer, right outer, and full outer joins.
5. Aggregations: Hive supports common aggregation functions such as COUNT, SUM, AVG, MIN, and MAX.
6. Subqueries: Hive supports subqueries in the WHERE and HAVING clauses.
7. Views: Views can be created in Hive to simplify complex queries and provide a level of abstraction.
8. UDFs: User-defined functions (UDFs) can be created and used in HiveQL queries to perform custom operations.
9. Optimization: Hive has several built-in optimization techniques, such as predicate pushdown and cost-based optimization, to improve query performance.
10. Execution Engines: Hive supports several execution engines, including MapReduce, Tez, and Spark, to process queries.

These are some of the key points to consider when querying data in Hive. It is important to have a good understanding of the data and the HiveQL language to write efficient and effective queries.

User Defined Functions in Hive

Hive is a data warehousing and SQL-like query language for Hadoop. It allows users to write custom functions, known as User Defined Functions (UDFs), to perform operations that are not available in the built-in functions.

Here are some key points to know about User Defined Functions in Hive:

1. UDFs can be written in Java and can be used in Hive queries.
2. UDFs can be used to perform complex data manipulations and transformations.
3. UDFs can be used to extend the functionality of Hive and provide custom solutions to specific problems.
4. UDFs can be created, registered, and used in Hive using the `CREATE FUNCTION` and `CREATE TEMPORARY FUNCTION` statements.
5. UDFs can be shared and reused by other users and queries.

Overall, User Defined Functions in Hive provide a powerful and flexible way to extend the capabilities of Hive and perform custom data manipulations and transformations. They are an essential tool for any advanced Hive user.

Sorting and Aggregating in Hive

Hive is a data warehousing tool built on top of Hadoop. It provides a SQL-like interface for querying and managing large datasets. Sorting and aggregating are two common operations performed on data in Hive.

1. **Sorting:** Sorting is the process of arranging data in a specific order. In Hive, you can use the `ORDER BY` clause to sort the data in ascending or descending order based on one or more columns. For example, to sort the data in a table named `employees` by the `salary` column in descending order, you can use the following query:

```
SELECT * FROM employees ORDER BY salary DESC;
```

2. **Aggregating:** Aggregation is the process of combining multiple rows of data into a single row, usually by performing some calculation on the data. In Hive, you can use aggregate functions such as `SUM`, `AVG`, `MIN`, `MAX`, and `COUNT` to perform calculations on the data. For example, to calculate the average salary of employees in a table named `employees`, you can use the following query:

```
SELECT AVG(salary) FROM employees;
```

You can also use the `GROUP BY` clause to group the data by one or more columns before performing the aggregation. For example, to calculate the average salary of employees in each department, you can use the following query:

```
SELECT department, AVG(salary) FROM employees GROUP BY department;
```

These are some of the basic concepts of sorting and aggregating data in Hive. You can use these operations to manipulate and analyze your data in various ways.

Map Reduce scripts in Hive

MapReduce is a programming model for processing large data sets in parallel across a distributed computing environment. Hive is a data warehousing and SQL-like query language for Hadoop, which facilitates reading, writing, and managing large datasets residing in distributed storage using SQL.

Here are some key points to remember when using MapReduce scripts in Hive:

1. Hive can generate MapReduce jobs automatically to execute SQL-like queries.
2. Hive supports custom MapReduce scripts through the `TRANSFORM` and `MAP/REDUCE` operators.

3. The **TRANSFORM** operator allows you to use custom scripts to transform the data as it is being processed by the MapReduce job.
4. The **MAP** and **REDUCE** operators allow you to specify custom Map and Reduce scripts to be used in the MapReduce job.
5. Custom MapReduce scripts can be written in any language that can read from standard input and write to standard output.
6. When using custom MapReduce scripts, it is important to ensure that the input and output formats are compatible with the data being processed.

Joins and Subqueries in Hive

Hive is a data warehousing and SQL-like query language for Hadoop, which allows users to perform operations on large datasets. One of the most powerful features of Hive is its ability to perform joins and subqueries.

1. **Joins:** Joins in Hive allow you to combine data from two or more tables based on a common column or condition. Hive supports several types of joins, including inner join, left outer join, right outer join, and full outer join.
2. **Subqueries:** Subqueries in Hive allow you to use the result of one query as input to another query. Subqueries can be used in various places within a query, such as in the **WHERE** or **HAVING** clause. Hive supports both correlated and uncorrelated subqueries.

Both joins and subqueries are powerful tools for data analysis and manipulation in Hive. They allow you to combine and analyze data from multiple tables, making it easier to gain insights and make data-driven decisions. It is important to understand how to use these features effectively to get the most out of your data in Hive.

HBase

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS).
- It provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases.
- HBase is well suited for real-time data processing or random read/write access to large volumes of data.
- It is ideal for high-scale real-time applications, such as a social media app or a streaming application.
- Thanks to the lack of a fixed database schema in a non-relational database like HBase, developers can add new data without conforming to a schema model.
- HBase is an open-source, distributed, versioned, non-relational database modeled after Google's Bigtable.
- It provides Bigtable-like capabilities on top of Hadoop and HDFS.

- HBase is developed as part of Apache Software Foundation's Apache Hadoop project and can run on top of HDFS (Hadoop Distributed File System) or Alluxio.

HBase Concepts

HBase is a distributed, column-oriented database that is built on top of the Hadoop Distributed File System (HDFS). It is an open-source project that is part of the Apache Hadoop ecosystem. Some of the key concepts of HBase include:

1. **Column Families:** HBase organizes data into column families, which are groups of related columns. Each column family is stored separately on disk, allowing for efficient access to specific columns.
2. **Rows:** HBase stores data in rows, with each row identified by a unique row key. Rows are sorted lexicographically by their row key, allowing for efficient range scans.
3. **Cells:** A cell is the intersection of a row and a column. It contains a value and a timestamp, which indicates when the value was last updated.
4. **Regions:** HBase automatically splits large tables into smaller units called regions, which are distributed across the cluster. This allows for horizontal scaling and efficient data access.
5. **Region Servers:** Region servers are responsible for managing regions. They handle read and write requests for the regions they manage and also perform compactions to merge multiple HFiles into one.
6. **HMaster:** The HMaster is responsible for coordinating the cluster. It assigns regions to region servers and handles load balancing and failover.
7. **WAL:** The Write-Ahead Log (WAL) is used to ensure data durability. When data is written to HBase, it is first written to the WAL before being written to the MemStore. In the event of a failure, the WAL can be used to recover data.
8. **MemStore:** The MemStore is an in-memory cache that stores data before it is flushed to disk. This allows for fast writes and low latency reads.
9. **HFile:** HFiles are the underlying storage format used by HBase. They are stored on HDFS and contain the actual data stored in HBase.

HBase Clients

HBase is a distributed, scalable, big data store that runs on top of the Hadoop Distributed File System (HDFS). It is a column-oriented database that is designed to handle large amounts of data across many commodity servers. HBase clients are used to interact with the HBase database.

Here are some key points to know about HBase clients:

1. HBase clients are used to perform operations on the HBase database, such as creating, reading, updating, and deleting data.
2. HBase clients can be written in various programming languages, including Java, Python, and Ruby.
3. The HBase client API is used to interact with the HBase database. The API provides methods for performing various operations on the database.
4. HBase clients can be used to perform batch operations, such as inserting or deleting multiple rows at once.
5. HBase clients can also be used to perform scans, which allow you to retrieve data from the database based on certain criteria.
6. HBase clients can be configured to use various levels of consistency, depending on the needs of the application.
7. HBase clients can be used in conjunction with other Hadoop ecosystem tools, such as MapReduce, to perform complex data processing tasks.

HBase Example

HBase is a distributed, column-oriented database that is built on top of the Hadoop Distributed File System (HDFS). It is an open-source project that is part of the Apache Hadoop ecosystem. HBase is designed to handle large amounts of data and is used for real-time read/write access to big data.

Here is an example of how to create a table in HBase:

1. Start the HBase shell by running the command `hbase shell`.
2. Create a table by running the command `create 'table_name', 'column_family'`. Replace `table_name` with the name of the table you want to create and `column_family` with the name of the column family.
3. Verify that the table was created by running the command `list`. This will show a list of all the tables in HBase.

This is a simple example of how to create a table in HBase. There are many other operations that can be performed in HBase, such as inserting data, querying data, and deleting data. These operations can be performed using the HBase shell or by using the HBase API in a programming language such as Java.

HBase vs RDBMS

- **RDBMS** (Relational Database Management System) and **HBase** are both types of database management systems, but they differ in several ways.
- **Data Model:** RDBMS uses a relational data model, where data is stored in tables with predefined columns and rows. HBase, on the other hand, uses a column-family data model, where data is stored in column families, which contain columns and rows. HBase is often referred to as a NoSQL database because of its non-relational data model.

- **Scaling:** RDBMS is more suitable for traditional, transactional applications that require strong consistency, whereas HBase is better suited for big data applications that require horizontal scaling and high-speed processing.
- **Consistency:** In HBase, there is no transaction guaranty, whereas RDBMS mostly guarantees transaction integrity.
- **JOINS:** HBase supports JOINS, whereas RDBMS does not support JOINS.
- **Referential integrity:** HBase does not guarantee referential integrity, whereas RDBMS does.
- **Speed:** HBase is built on Hadoop and modeled after Google BigTable. HBase provides random access and strong consistency for large amounts of unstructured and semi-structured data in a schemaless database organized by column families.

Advanced Usage of HBase

HBase is a distributed, scalable, big data store that provides random, real-time read/write access to large datasets. Here are some advanced usage tips for HBase:

1. **Data Modeling:** HBase data modeling is different from traditional relational databases. It is important to design the row key and column families carefully to optimize performance and storage efficiency.
2. **Compression:** HBase supports several compression algorithms, including GZ, LZO, LZ4, and Snappy. Using compression can significantly reduce the amount of disk space required to store data and improve read performance.
3. **Bloom Filters:** Bloom filters can be used to reduce the number of disk lookups required to find a row. This can significantly improve read performance for certain workloads.
4. **Coprocessors:** Coprocessors are custom code that can be executed on the server side to perform advanced data processing. They can be used to implement custom aggregation, filtering, and transformation logic.
5. **Bulk Loading:** HBase provides a bulk loading tool that can be used to efficiently load large amounts of data into a table. This can be much faster than using the standard API to insert data one row at a time.
6. **Compaction:** HBase periodically performs compactions to merge multiple HFiles into a single file. This can improve read performance and reduce the amount of disk space required to store data. It is important to monitor and tune compaction settings to ensure that compactions do not negatively impact performance.
7. **Region Splitting:** HBase automatically splits regions when they become

too large. It is important to monitor and tune the split policy to ensure that regions are split at the appropriate size to balance performance and resource usage.

8. **Backup and Disaster Recovery:** HBase provides several tools for backup and disaster recovery, including snapshots, replication, and export/import. It is important to have a well-defined backup and disaster recovery plan to ensure data durability and availability.

These are some advanced usage tips for HBase that can help improve performance, efficiency, and reliability. It is important to carefully design and tune HBase deployments to meet the specific needs of the application and workload.

Schema Design in HBase

- HBase schema design is very different compared to the relation database schema design .
- HBase does not support any kind of joins, but it provides the single-indexing strategy on the row key .
- Each table in HBase table is indexed on row key. Data is sorted lexicographically by this row key .
- HBase schema design supports denormalization with nested entities .
- These nested entities are nothing but a column whose name is the unique identifier for the nested entity and whose value is the entire record mashed together .
- Since HBase allows dynamic column definition, there's no problem .

Advanced Indexing in HBase

- HBase is a column-oriented NoSQL database management system that runs on top of the Hadoop Distributed File System (HDFS). It is modeled after Google's Big Table and written in Java.
- In HBase, there are no indexes. The rowkey, column family, column qualifier are all stored in sort order based on the java comparable method for byte arrays.
- Access to records in any way other than through the primary row key requires scanning over potentially all the rows in the table to test them against your filter.
- Secondary indexing is a way to improve the performance of queries that do not use the primary row key. HBASE-9203 is a Jira entry that exists specifically to address the ideas behind secondary indexing.
- An index will surely work faster than scanning a large number of rows every time. If you use an HBase version that already has coprocessors you can follow the advice given in the documentation. If you are using older versions of HBase you need to set up an additional table to act as the index and update manually.

Zookeeper

- A zookeeper is a professional who is responsible for the care and management of animals in a zoo.
- Zookeepers are responsible for the daily care of animals, including feeding, cleaning, and monitoring their health and behavior.
- They also design and maintain animal habitats, ensuring that the animals have a safe and comfortable environment to live in.
- Zookeepers work closely with veterinarians to provide medical care to the animals when needed.
- They also play a role in educating the public about the animals and their conservation.
- To become a zookeeper, one typically needs a degree in a related field such as zoology, biology, or animal science, as well as experience working with animals.
- The job of a zookeeper can be physically demanding and may require working outdoors in all weather conditions.
- Zookeepers must also be able to communicate effectively with colleagues and the public, and have a strong passion for animal welfare and conservation.

Zookeeper concepts

Apache ZooKeeper is a distributed coordination service that enables distributed systems to coordinate with each other through a shared hierarchical namespace. Some of the key concepts of ZooKeeper are:

1. **Znodes:** ZooKeeper stores data in a hierarchical namespace, similar to a file system. Each node in the namespace is called a znode. Znodes can store data and have children.
2. **Data Model:** ZooKeeper's data model is a tree of znodes, where each znode can have data associated with it and can have children znodes.
3. **Watches:** Clients can set watches on znodes. A watch is a one-time trigger that notifies the client when the data of the watched znode changes.
4. **Ephemeral Nodes:** ZooKeeper supports ephemeral nodes, which are znodes that exist as long as the session that created them is active. When the session ends, the ephemeral nodes are automatically deleted.
5. **Sequential Nodes:** ZooKeeper supports sequential nodes, which are znodes that have a monotonically increasing sequence number appended to their name. This is useful for implementing distributed locks and queues.
6. **Access Control:** ZooKeeper supports access control through Access Control Lists (ACLs), which specify the operations that different users or groups of users are allowed to perform on a znode.

7. **Consistency Guarantees:** ZooKeeper provides strong consistency guarantees, including linearizable writes and FIFO client order.

These are some of the key concepts of ZooKeeper. It is a powerful tool for building distributed systems and provides a simple and robust foundation for coordination and synchronization.

How Zookeeper helps in monitoring a cluster

Zookeeper is a tool that helps in maintaining configuration information, naming, and group services for distributed applications. It implements different protocols on the cluster so that the application should not implement on their own. It provides a single coherent view of multiple machines.

Some of the ways in which Zookeeper helps in monitoring a cluster are:

1. **Maintaining consistency:** ZooKeeper hosts are deployed in a cluster and, as long as a majority of hosts are up, the service will be available. Applications Manager's ZooKeeper monitoring helps make sure the total node count inside the ZooKeeper tree is consistent.
2. **Thread and JVM usage:** Zookeeper monitoring can help in analyzing a JVM Thread Dump and pinpoint the root cause of issues.
3. **Monitoring progress of distributed data:** As an example, HBase makes use of Apache ZooKeeper to monitor the progress of distributed data.
4. **Prometheus monitoring service:** Running a Prometheus monitoring service is the easiest way to ingest and record ZooKeeper's metrics.

There are several tools available for Zookeeper monitoring, such as Sematext, Prometheus & Grafana, ManageEngine, Site24x7, OpsView, Instana, SignalFx, Datadog, and AppDynamics.

How to build applications with Zookeeper

1. **Download a stable ZooKeeper release:** To get a ZooKeeper distribution, download a recent stable release from one of the Apache Download Mirrors.
2. **Setting up a ZooKeeper server:** Setting up a ZooKeeper server in standalone mode is straightforward. The server is contained in a single JAR file, so installation consists of creating a configuration.
3. **Starting ZooKeeper and Application Builder:** To start ZooKeeper and Application Builder after installing Application Builder in admin mode: (Linux) - On the server (s) where ZooKeeper is installed, execute the `/etc/init.d/zookeeper-service-default`. To start ZooKeeper and Application Builder after installing Application Builder in non-admin mode: Start the ZooKeeper server before starting Application Builder. To start the ZooKeeper server on a Linux system, use the

Zookeeper/zookeeper/bin/zkServer.sh restart command from your Watson Explorer installation directory.

4. **Running ZooKeeper on Kubernetes:** Open a terminal, and use the kubectl apply command to create the manifest. `kubectl apply -f https://k8s.io/examples/application/zookeeper/zookeeper.yaml` This creates the zk-hs Headless Service, the zk-cs Service, the zk-pdb PodDisruptionBudget, and the zk StatefulSet.
5. **Using ZooKeeper to build distributed apps:** ZooKeeper is already used by Apache HBase, HDFS, and other Apache Hadoop projects to provide highly-available services and, in general, to make distributed programming easier. You can use ZooKeeper to easily and safely implement important features in your distributed software.

IBM Big Data strategy

IBM, a US-based computer hardware and software manufacturer, had implemented a Big Data strategy, where the company offered solutions to store, manage, and analyze the huge amounts of data generated daily and equipped large and small companies to make informed business decisions .

- IBM's Big Data strategy is part of its corporate initiative called Smarter Planet, which sought to highlight how government and business leaders were capturing the potential of smarter systems to achieve economic and sustainable growth and societal progress.
- IBM's data strategy framework consists of six steps: understanding business objectives, assessing the current state, mapping out the data strategy framework, defining data's target state, identifying and prioritizing initiatives, and executing and governing the data strategy.
- IBM also offers Big Data Analytics solutions, driving advanced analytics with an enterprise-grade, secure, governed, open source-based data lake.
- IBM recommends giving data assets and accelerators top priority, developing a process and culture around data that enables true standardization, re-use, portability, speed to action, and risk reduction across the end-to-end data lifecycle.

IBM Big Data strategy

IBM, a US-based computer hardware and software manufacturer, had implemented a Big Data strategy. The company offered solutions to store, manage, and analyze the huge amounts of data generated daily and equipped large and small companies to make informed business decisions .

- IBM's Big Data strategy is part of its corporate initiative called Smarter Planet, which sought to highlight how government and business leaders were capturing the potential of smarter systems to achieve economic and sustainable growth and societal progress.

- IBM's data strategy framework consists of six steps: understanding business objectives, assessing the current state, mapping out the data strategy framework, defining data target architecture, building a data-driven culture, and measuring success.
- IBM also offers Big Data analytics solutions, such as an enterprise-grade, secure, governed, open source-based data lake, and partnerships with companies like Cloudera to connect the data lifecycle and accelerate the journey to hybrid cloud and AI.
- IBM recommends giving data assets and accelerators top priority, developing a process and culture around data that enables standardization, re-use, portability, speed to action, and risk reduction across the end-to-end data lifecycle.

Introduction to Infosphere

- Infosphere is a term that combines “information” and “sphere” to describe a metaphysical realm of information, data, knowledge, and communication .
- This realm is populated by informational entities called inforgs or informational organisms .
- IBM InfoSphere Information Server is a software platform that provides a single platform for data integration and governance .
- The components in the suite combine to create a unified foundation for enterprise information architectures, capable of scaling to meet any information volume requirements .
- IBM InfoSphere Streams is another software platform that enables the development and execution of applications that process information in data streams .
- InfoSphere Streams enables continuous and fast analysis of massive volumes of moving data to help improve the speed of business insight and decision making .

Introduction to BigInsights

BigInsights is an IBM distribution of Apache Hadoop, a software framework for distributed processing of large data sets across clusters of computers. BigInsights includes several IBM value-added components, such as Big SQL, BigSheets, and Text Analytics, that enhance the capabilities of Apache Hadoop.

Some key features of BigInsights include: - Scalability: BigInsights can handle petabytes of data and thousands of nodes. - Flexibility: BigInsights can process structured, semi-structured, and unstructured data. - Cost-effectiveness: BigInsights can run on commodity hardware, reducing the cost of ownership. - Ease of use: BigInsights includes several tools and interfaces that make it easier for users to interact with and analyze data.

BigInsights is used in a variety of industries, including finance, healthcare, and retail, to analyze large amounts of data and gain insights into customer behavior, market trends, and operational efficiency. It is a powerful tool for businesses looking to harness the power of big data.

Introduction to Big Sheets

Big Sheets is a web-based tool that allows users to analyze and visualize large amounts of data. It is designed to handle data sets that are too large to be easily manipulated in traditional spreadsheet programs like Microsoft Excel.

Some key features of Big Sheets include:

- The ability to handle large data sets: Big Sheets can handle millions of rows of data, making it ideal for working with large data sets.
- Data visualization: Big Sheets includes a variety of visualization tools that allow users to create charts, graphs, and other visual representations of their data.
- Collaboration: Multiple users can work on the same Big Sheet at the same time, making it easy to collaborate on data analysis projects.
- Integration with other tools: Big Sheets can be integrated with other data analysis tools, allowing users to import data from other sources and export their results for further analysis.

Overall, Big Sheets is a powerful tool for anyone who needs to work with large amounts of data. Its ability to handle large data sets, combined with its visualization and collaboration features, make it an ideal choice for data analysis projects.

Introduction to Big SQL

Big SQL is a high-performance SQL engine that is used to query and analyze data stored in Hadoop. It is a hybrid SQL engine that combines the power of Hadoop with the familiarity of SQL. Some of the key features of Big SQL include:

1. **SQL Compatibility:** Big SQL is compatible with the SQL-2011 standard and supports a wide range of SQL functions and data types.
2. **Performance:** Big SQL is designed to deliver high performance for complex queries on large data sets. It uses advanced query optimization techniques and can take advantage of the parallel processing capabilities of Hadoop.
3. **Integration with Hadoop:** Big SQL is tightly integrated with Hadoop and can access data stored in HDFS, HBase, and other Hadoop data sources.
4. **Scalability:** Big SQL can scale to handle large data sets and high query volumes. It can be deployed on a cluster of commodity hardware and can take advantage of the distributed processing capabilities of Hadoop.

5. **Security:** Big SQL provides robust security features, including authentication, authorization, and data encryption.

Overall, Big SQL is a powerful tool for querying and analyzing data stored in Hadoop. It provides the familiarity of SQL with the performance and scalability of Hadoop.